

# AI & ML Systems Playbook

*RAG, agents and the model-training stack behind production AI systems.*

19 chapters · compiled from the author's engineering notes and essays

By Pradyumna Chippigiri

Compiled 4 October 2026

## WHAT'S INSIDE

- Zero to One : Learning Agents and Agentic Patterns
- Everything you need to know about production ready RAG systems : Part 1
- Everything you need to know about Production ready RAG systems: Part 2
- Everything you need to know about production ready RAG systems : Part 3
- A Supa Explanation of Supabase: The BaaS Everyone Is Talking About (And Why)
- Agent Memory Types
- How Agents write to external long term memory
- Prompt Injection
- Knowledge Distillation
- Proper distillation - type of knowledge distillation
- Model parallelism
- Distributed Data Parallelism
- Strategies for scaling neural networks across multiple GPUs
- LLM Compute constraints
- Logits
- Vanishing Gradient
- ReLu activation function
- Sigmoid activation function
- Tanh activation function

|                                                                                                                         |   |
|-------------------------------------------------------------------------------------------------------------------------|---|
| <b>Everything you need to know about production ready RAG systems : Part 1</b>                                          | 5 |
| <i>From foundational building blocks to multi-agent systems: A deep dive into agentic patterns and memory in agents</i> |   |
| <b>Zero to One : Learning Agents and Agentic Patterns</b>                                                               | 3 |

## CONTENTS

# Zero to One : Learning Agents and Agentic Patterns

2026-01-21 · From foundational building blocks to multi-agent systems: A deep dive into agentic patterns and memory in agents

Source: <https://pradyumnachippigiri.substack.com/p/zero-to-one-learning-agents-and-agentic>

---

I have been working on AI Agents for some time now, and upskilling myself in this space.

As I learn I have been documenting everything from various references, and sources.

This turned into a pretty large knowledge dump covering concepts, architectures, code, and diagrams, so it didn't fit nicely into a Substack post. That's why I'm linking out for this article. Access the link below. n

LINK ? : <https://pradyumnachippigiri.dev/blogs/understanding-ai-agents>

---

Below is the table of contents so you can quickly see what's covered.

| TABLE OF CONTENTS                         |
|-------------------------------------------|
| <a href="#">Overview</a>                  |
| <a href="#">What are AI Agents?</a>       |
| <a href="#">Agents vs Workflows</a>       |
| <a href="#">Agent Usecases</a>            |
| <a href="#">Building Blocks of Agents</a> |
| <a href="#">LLMs</a>                      |
| <a href="#">Tools</a>                     |
| <a href="#">Memory in Agents</a>          |
| <a href="#">Short-term Memory (STM)</a>   |
| <a href="#">Long-term Memory (LTM)</a>    |
| <a href="#">Memory Management</a>         |
| <a href="#">ReACT Pattern</a>             |
| <a href="#">Agentic Frameworks</a>        |
| <a href="#">Workflow Patterns</a>         |
| <a href="#">Prompt Chaining</a>           |
| <a href="#">Routing</a>                   |
| <a href="#">Parallelization</a>           |
| <a href="#">Orchestrator-Workers</a>      |
| <a href="#">Agentic Patterns</a>          |
| <a href="#">ReACT Pattern</a>             |
| <a href="#">Reflection Pattern</a>        |
| <a href="#">Tool Use Pattern</a>          |
| <a href="#">Multi Agent pattern</a>       |
| <a href="#">Multi Agent Patterns</a>      |
| <a href="#">Subagents Pattern</a>         |
| <a href="#">Skills Pattern</a>            |
| <a href="#">Handoffs Pattern</a>          |
| <a href="#">Router Pattern</a>            |
| <a href="#">More of my writing in AI</a>  |
| <a href="#">Next Steps</a>                |
| <a href="#">References</a>                |

Here's a list of references and sources I have learned from:

## References

- [Google Cloud Tech Youtube Videos](#)
- [Anthropic's Guide to building effective Agents](#)
- [Open AI's Practical guide to building Agents](#)
- [Microsoft: AI Agent for beginners](#)
- [IBM: Guide to Ai Agents](#)
- [Vizuara AI Labs and Dr. Raj Dandekar](#)
- [Andrew Ng: Agentic AI Course](#)
- [Phil Schmid: Rise of Subagents](#)
- [Aman: Primer on AI Agents](#)
- [Leonie Monigatti: Making sense of memory in Agents](#)
- [Cognition: Dont build multi-agents](#)
- [Simon Willison: AI Agents Definitions](#)
- [Agentlearn.dev](#)
- [Phil Schmid: Memory in AI Agents](#)
- [Weaviate: What are Agentic Workflows?](#)
- [Anthropic: Multi Agent Research System](#)
- [Langchain: Choosing the right multi-agent architectures](#)
- [DailyDoseOfDS, Avi Chawla : Ai Agents guidebook](#)
- [Lance Martin: Context Engineering for Agents](#)
- [Drew Breunig: Why Context Engineering Matters](#)
- [Tina Huang : AI Agents video](#)
- [Letta Blogs : Agent Memory, How to build Ai Agents that learn and remember](#)
- [LangChain + LangGraph Docs: Memory](#)
- [Aishwarya Naresh Reganti: LevelUpLabs free AI Agent course on Github](#)

Here's a short preview video below so you can quickly see how the article is structured.

---

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

Thanks for reading Evolving Engineer! If you know someone learning agents, feel free to share it. If this post helped, you can support my work here - Buy Me a Coffee ?

See you in the next one ?.

# Everything you need to know about production ready RAG systems : Part 1

2025-11-18 · Deep-diving into document ingestion, chunking strategies, and the architecture of real-world retrieval systems

Source: <https://pradyumnachippigiri.substack.com/p/everything-you-need-to-know-about>

---

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/rag-systems-part1>

## What is RAG?

RAG stands for Retrieval Augmented Generation. It was introduced in the paper Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

Each step can be roughly broken down to:

Retrieval – Seeking relevant information from a source given a query. For example, getting relevant passages of Wikipedia text from a database given a question.

Augmented – Using the relevant retrieved information to modify an input to a generative model (e.g., an LLM).

Generation – Generating an output given an input. For example, in the case of an LLM, generating a passage of text given an input prompt.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

You can think of RAG like an open-book exam:

You or your mind is the LLM.

The textbook is your external knowledge base (PDFs, docs, webpages).

When a question is asked, you don't memorize the entire book, instead, you first flip through the index/table of contents to find the most relevant pages -> This "flipping and finding the right pages" is Retrieval.

Once you have the right pages open, you read them and augment your understanding with that information -> This is Augmentation.

Finally, based on the question + the retrieved pages, you think, synthesize, and produce a well-structured answer -> This is Generation.

So the pipeline is:

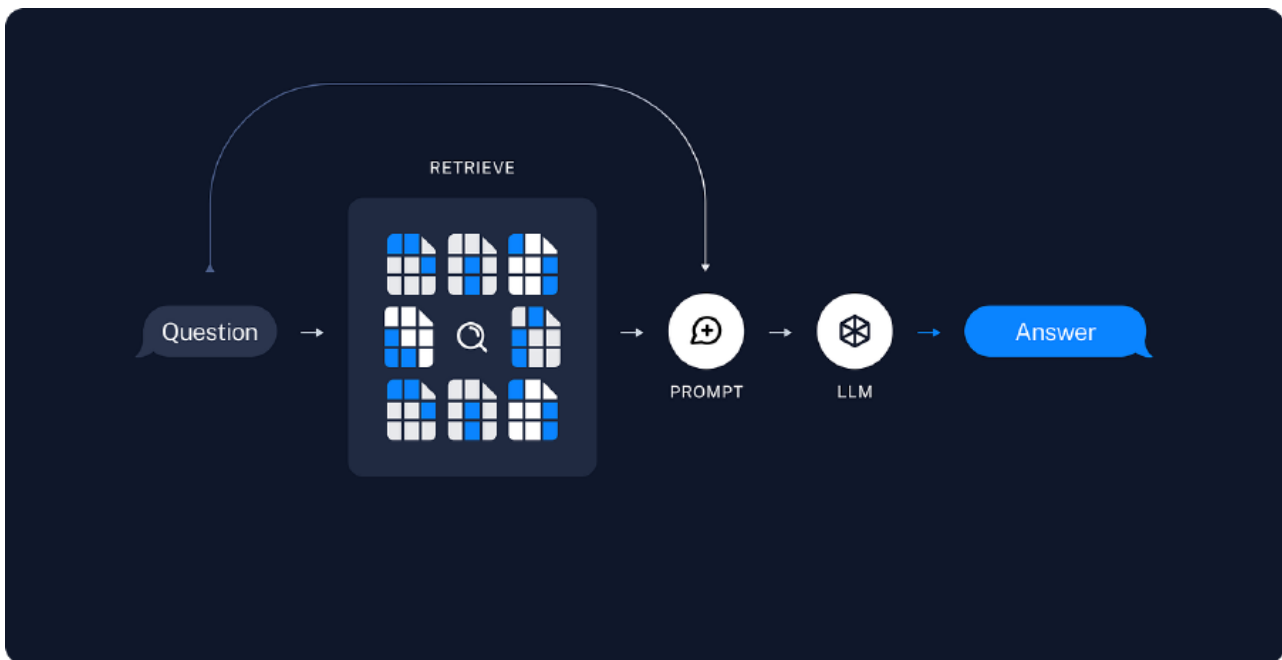
Index search -> retrieve relevant pages -> read them -> think -> answer.

Exactly how RAG works:

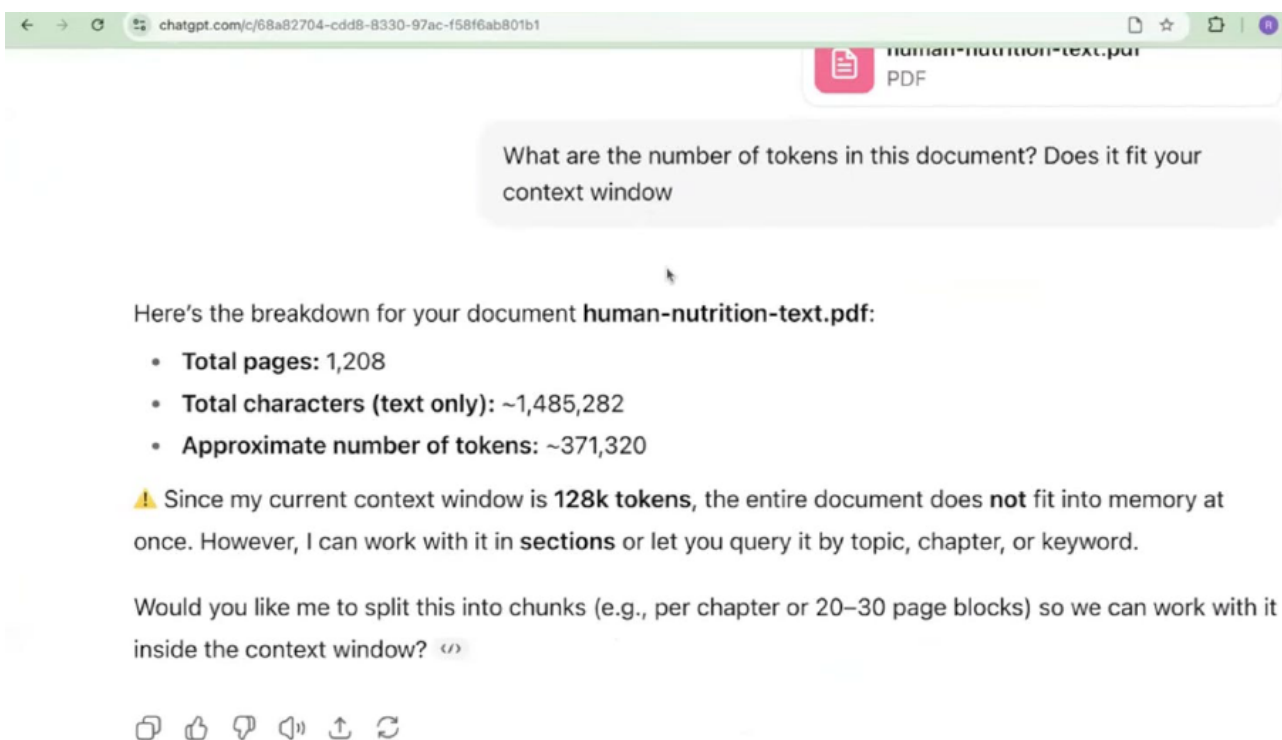
Retriever: finds relevant chunks like an exam index

Augmentor: inserts those chunks into the model's prompt

LLM Generator: reasons using the question + retrieved context



You might now ask, our mind is not capable to read the entire book to answer quickly to the question, but LLMs can right, so what is the use of RAG ? I asked ChatGPT and Google Gemini to answer questions from a 1000 page pdf and here's how it answered.



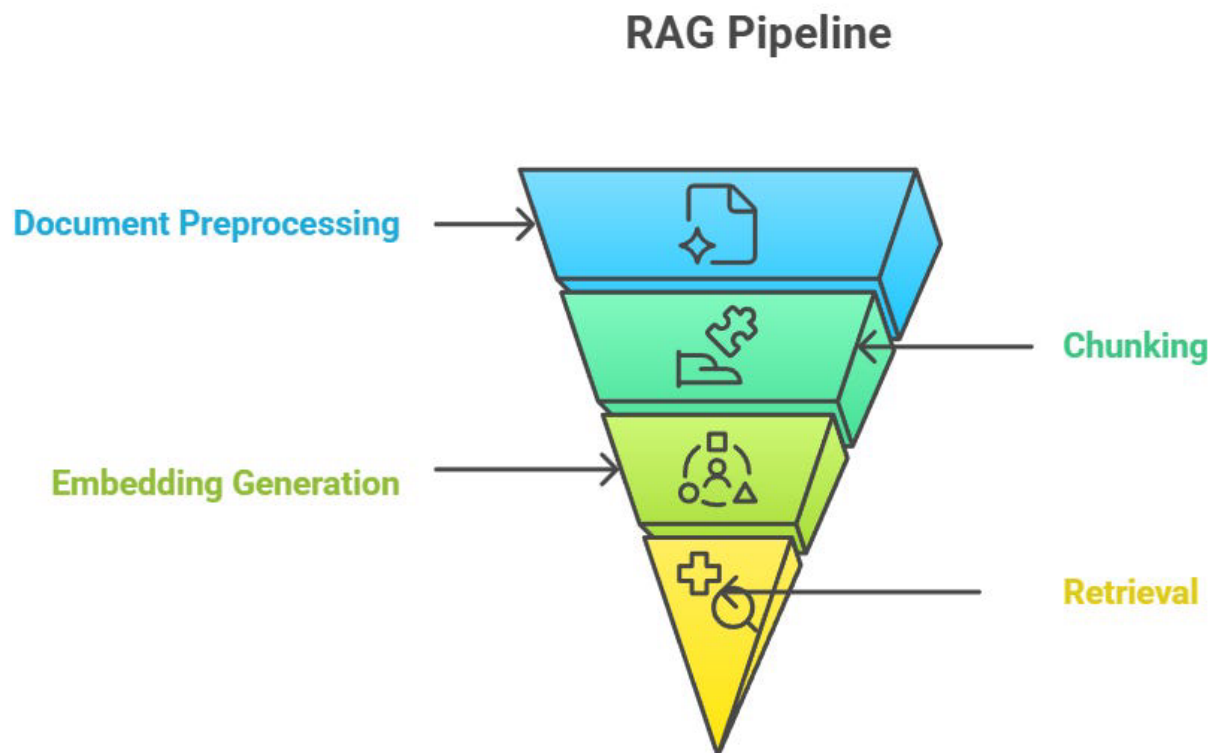
As you can see that Chatgpt was not able to answer as its context window is lesser compared to Gemini. Hence, Google Gemini was able to read and output the question asked. But, here comes the catch: Even though LLMs today have massive context windows and can technically “read” huge amounts of text, there are three major limitations that make RAG still essential.

Even though LLMs today have massive context windows and can technically “read” huge amounts of text, there are two major limitations that make RAG still essential. - Cost explodes with large context windows: LLMs charge per input + output token. Sending a 200K-token PDF into the model every time is insanely expensive.

- Attention quality drops as context increases: LLMs lose accuracy and focus when you overload them. RAG narrows the context -> better reasoning. So RAG is valuable and will be in the future too

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

## The Four Main Building Blocks of a RAG System



In this article we will talk about Document Preprocessing that is basically Data Ingestion, and Data Chunking and different strategies in chunking. In the next article we will talk about Embedding generation and retrieval.

### Data Ingestion

Everyone thinks Data Ingestion is easy “just upload a PDF and extract text” but there are so many complexities involved in this as well, If given a PDF, where would you store it, how would you parse it? How would you deal with images and tables?

There are some powerful libraries that you should know:

PyMuPDF (fitz) Best for mixed PDFs (text + images). Extracts:

text

images

metadata

layout

Docs: <https://pymupdf.readthedocs.io/en/latest/>

But if a page is JUST a scanned image, PyMuPDF cannot read text -> you need OCR.

OCR using Tesseract For scanned PDFs or images:

Reads text from images

Works for scanned PDFs

Not ideal for complex tables or multi-column layouts

Repo: <https://github.com/tesseract-ocr/tesseract>

Docling Extracts:

rows

columns

table schemas

layout

and converts everything into clean JSON

Docs: <https://docling-project.github.io/docling/>

These libraries help you in opening and reading the pdfs, but in some cases when you are dealing with industry level projects, often you do not have the pdfs, you will have to scrape the data from the web. So here are some libraries you would want to know to scrape data from the web.

Firecrawl Fully crawls:

websites

blogs

documentation

knowledge bases

...and converts them into LLM-ready Markdown.

Docs - <https://www.firecrawl.dev/>

Puppeteer Think of it as an automated browser that you can script the same way you would manually click, search, scroll, and interact with web pages. Also lets you scrape only what you want:

tags

headers

titles

selected divs

links to all PDFs

and even download 500 PDFs automatically

Docs - <https://pptr.dev/>

## Data Chunking

Once your documents are cleaned and extracted, the next question is: "How should I split the text so the LLM can retrieve the right information?"

Chunking matters because:

too large = irrelevant info

too small = lost context

badly aligned = wrong retrieval

There are five chunking strategies, each with specific use cases and each of them have their own pros and cons, lets discuss all of that in this discussion :

**Fixed-Size Chunking** Fixed-size chunking divides text into chunks based purely on length, for example, every 200 words or every 500 tokens. It simply chops the text based on size rules, which makes it extremely fast but contextually weak.

Pros:

Very fast and easy to implement

Works well for large volumes of short, noisy text

Cons:

Breaks semantic flow (chunks may start mid-sentence)

Context can get lost across boundaries

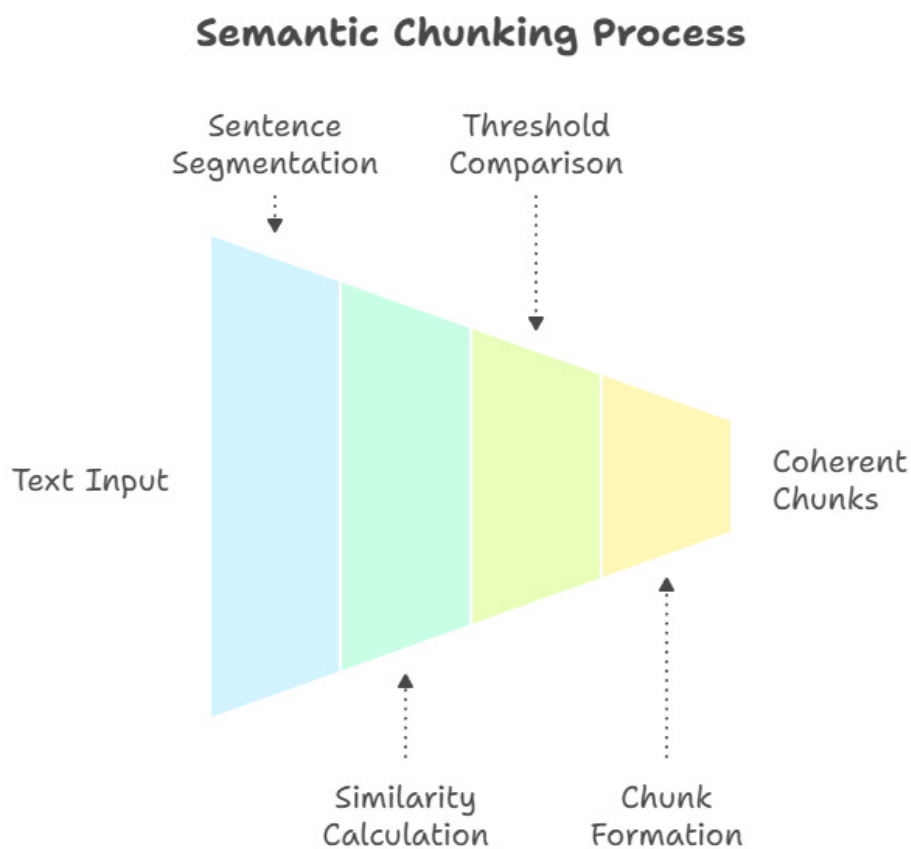
Best For:

Social media data (reddit, twitter etc.)

Logs

Short, unstructured content where coherence is less important

Semantic Chunking



Semantic chunking uses embedding similarity to group sentences that are meaningfully related. Instead of breaking text by size, it breaks based on content. Each sentence is compared to the chunk being built; if it stays semantically similar, it joins the chunk. If it drifts too far than the cosine similarity threshold that we mention then, a new chunk is created.

Pros:

Produces highly coherent, context-rich chunks

Great for deep retrieval accuracy

Cons:

Slow (needs embeddings for every sentence)

Chunk sizes can vary (some can be tiny, others huge)

Best For:

Legal contracts

Policies, compliance docs

**Structural Chunking** Structural chunking uses the natural format of the document headings, subheadings, sections, tables of contents to define chunk boundaries. It respects the author's intended organization, making chunks more meaningful and human-readable. Example by Introduction, Overview, Results etc.

Pros:

Very clean, interpretable chunks

Fast and consistent

Preserves metadata like titles and sections

Cons:

Sections may vary widely in length hence chunk sizes can vary immensely.

Not ideal for documents with poor or missing structure

Best For:

PDFs with headings

Research papers

Reports, guides, manuals

Technical Documentations

**Sliding-window chunking** Sliding-window chunking creates overlapping chunks to preserve continuity. Instead of splitting text once, you slide a window across the document for example, a 500-token window moving forward 250 tokens at a time. This ensures that important context that appears near chunk boundaries is not lost.

Example: Chunk 1: tokens 0–500 Chunk 2: tokens 250–750 Chunk 3: tokens 500–1000

Pros:

Maintains continuity across chunk boundaries

Reduces “edge-case” information loss

Improves retrieval for sequential content

Cons:

Higher storage requirements

More redundant data to embed and store

Best For:

Transcripts

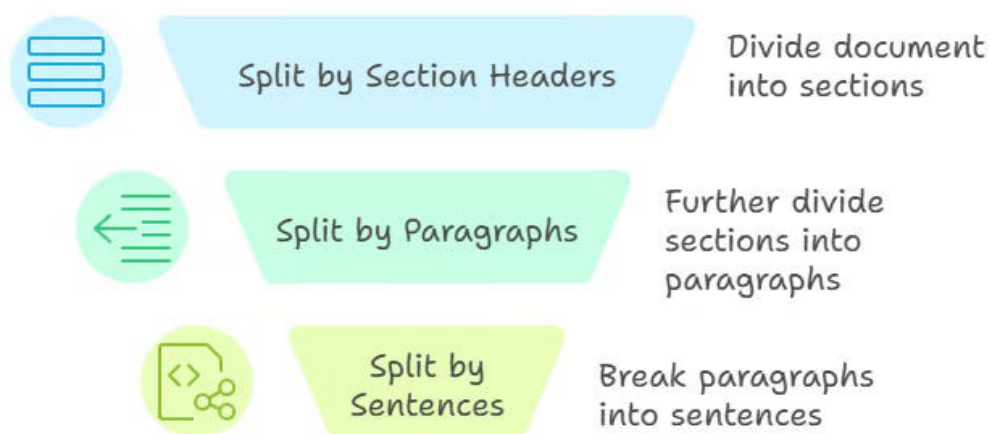
Podcasts

Dialogue/conversations

Any long, sequential content

Recursive chunking

## Recursive Chunking Process



Recursive chunking is a hybrid strategy. It first splits the document using high-level structure (e.g., headings). Then, if any section exceeds a maximum size that we mention, it splits that section again using another method (often fixed-size or semantic). This maintains both structure and consistency.

Pros:

Produces consistently sized chunks

Still respects hierarchy and structure

Ideal for large, messy documents

Cons:

More complex to implement

Requires choosing multiple chunking rules

Best For:

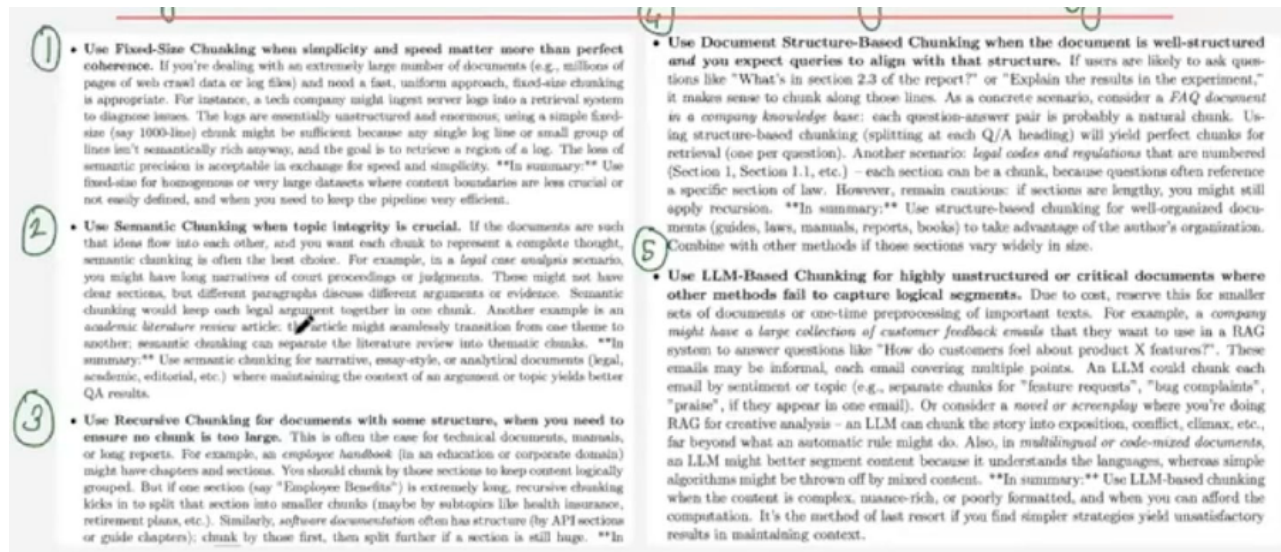
Enterprise-grade RAG

Long, heterogeneous PD

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

## Which chunking strategy to use and when ?

The image below shows exactly when we should prefer one over the other.



the most important question to ask is: "Does my data need chunking at all?"

Chunking is designed to break down long, unstructured documents. If your data source already has small, complete pieces of information like FAQs, product descriptions, or social media posts, you usually do not need to chunk them. Chunking can even cause problems. The goal is to create meaningful semantic units, and if your data is already in that format, you're ready for the embedding stage.

Once you've confirmed that your documents are long enough to benefit from chunking, you can use the following questions to guide your choice of strategy:

What is the nature of my documents? Are they highly structured (like code or JSON), or are they unstructured narrative text?

What level of detail does my RAG system need? Does it need to retrieve specific, granular facts or summarize broader concepts?

Which embedding model am I using? What are the size of the output vectors (more dimensions increases the ability for more granular information to be stored)??

How complex will my user queries be? Will they be simple questions that need small, targeted chunks, or complex ones that require more context?

## How to Optimize The Chunk Size for RAG in Production

Optimizing chunk size in a production setting takes many tests and reviews. Here are some steps you can take:

Begin with a common baseline strategy, such as fixed-size chunking. A good place to start is a chunk size of 512 tokens and a chunk overlap of 50-100 tokens. This gives you a solid baseline that's easy to reproduce and compare other chunking strategies against.

Experiment with different chunking approaches by tweaking parameters like chunk size and overlap to find what works best for your data.

Test how well your retrieval works by running typical queries and checking metrics like hit rate, precision, and recall to see which strategy delivers.

Involve humans to review both the retrieved chunks and LLM-generated responses - their feedback will catch things metrics might miss.

Continuously monitor the performance of your RAG system in production and be prepared to iterate on your chunking strategy as needed.

Already this article has gone sorry for that, in the next article we will learn about Embeddings and Generation part of it. Hope you liked this article, and if you liked it please do leave a like. See you in the next one.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

# Everything you need to know about Production ready RAG systems: Part 2

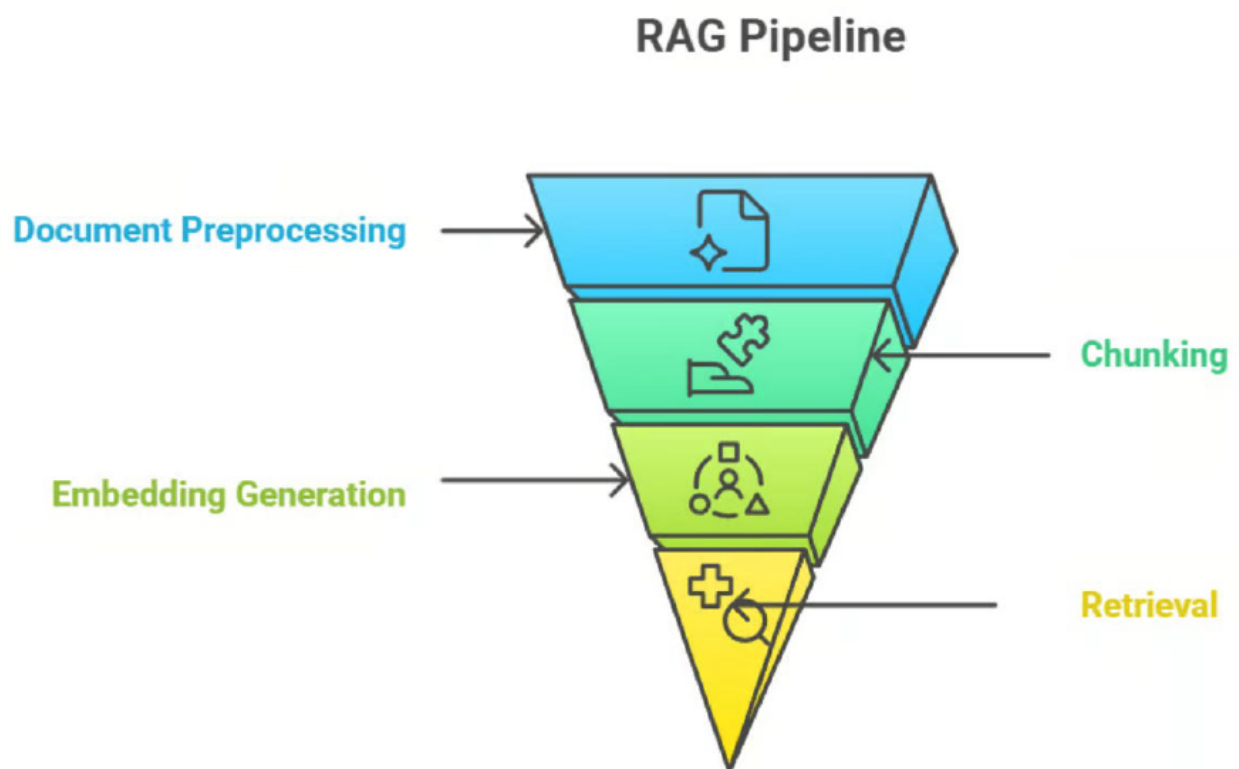
2025-11-23 · Building the semantic layer of RAG: embeddings, vector stores, and high-speed retrieval pipelines

Source: <https://pradyumnachippigiri.substack.com/p/everything-you-need-to-know-about-f99>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/rag-systems-part2>

In Part 1 of this series, we talked about data ingestion and chunking and we learnt how to get raw data into the system and split it into pieces using different chunking strategies.



In this part, we'll cover the last two core pieces of the RAG pipeline:

Embedding generation – turning text into vectors that carry meaning.

Retrieval – using those vectors to fetch the right chunks at query time.

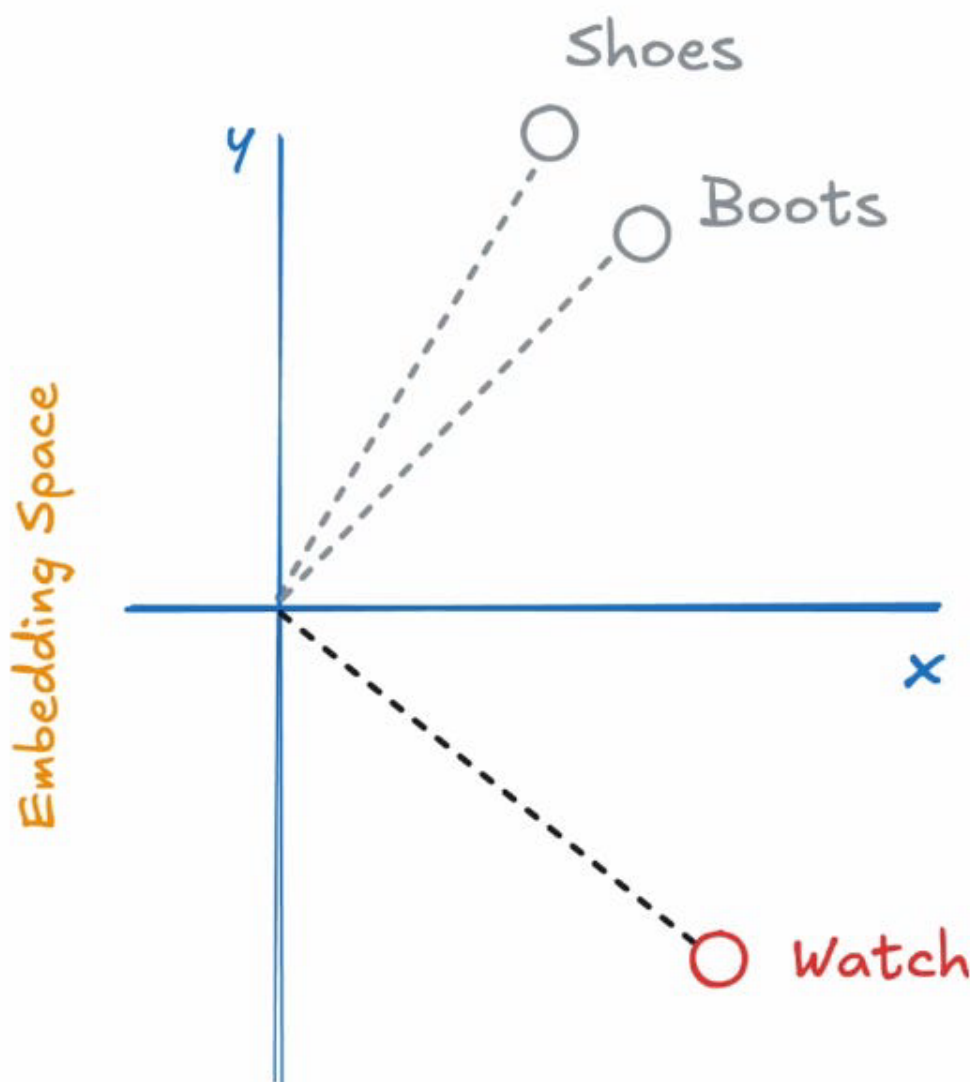
In Part 3 we will talk about the Generation step in RAG, along with different Evaluation strategies to evaluate the setup of the RAG pipeline.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

## Embeddings: turning text into meaningful vectors

In simple terms, embeddings are numerical representations of data. These numerical representations are not random; they carry some meaning. These embeddings help in capturing the semantic meaning and similarity between data points. For instance, the words “shoes” and “boots” are related in a way that “watch” is not. Embedding is not just for text; it can be applied to images, audio, or even graph data as well. There are multiple embedding techniques, I can make an entire series on that. But for now to understand what makes good embedding, these are the 2 points :

Semantic similarity - This means that words with closer meanings or relationships are closer in the vector space than words that are less related. It uses the concept of cosine similarity.



Dimensions - What should be the size of an embedding vector ? Striking the right balance is key because there are trade offs for example lower lower-dimensional vectors are more efficient to keep in memory or to process, while higher-dimensional can capture intricate relationships, but are prone to overfitting.

From these embedding models families, sentence transformers are one such; sentence transformers enable the transformation of sentences into vectors in vector spaces. They represent sentences as dense vector embeddings. Traditional embedding models used to work on word-level, and sentence transformers work on a sentence level.

Some popular sentence transformers are :

Closed(-ish) models via API

These are the ones you call over an API :

OpenAI (text-embedding-3-large) - OpenAI Embedding Docs

Google Gemini (gemini-embedding-001) - Gemini Embedding Docs

You can't download the model or fine-tune it locally. You just send text -> get back embeddings/vectors. Think of them as black box but convenient.

Open(-weight) models you can run yourself

These are models where you can download the weights and run them locally or on your own cloud:

Sentence-transformers models on Hugging Face(all-MiniLM-L6-v2, all-mpnet-base-v2, etc.)

Gemma, LLaMA, Mistral, etc.

In these kind of models you have more control (privacy, cost, custom infra) You can refer to this link from Huggingface to see the topmost sentence transformers -> Sentence Transformers Leaderboard Embedding models like Gemini, OpenAI can take in huge paragraphs to compute the vectors as they are huge embedding models. (most of these models are multimodals they can deal with images and table etc. ) How do you decide which embedding models to use for your RAG usecase ? So to summarise these are the factors to be considered :

Size of the input : If longest input sequences are to be embeded, then chose a model with larger input capacity.

size of embedding vector - Like we discussed larger dimensionalities are more accurate but need more compute and storage.

size of model - Larger models are better but again require more compute and power

open or closed - open models allow to run on your own hardware and closed are easier to setup but require API calls to get the embeddings.

Coming back to RAG, now that we have the chunked documents based on the best chunking strategy for that particular use case, we take the embedding of these chunks. But where do we store these embeddings of the document chunks?

Thanks for reading LevelYouUp! This post is public so feel free to share it.

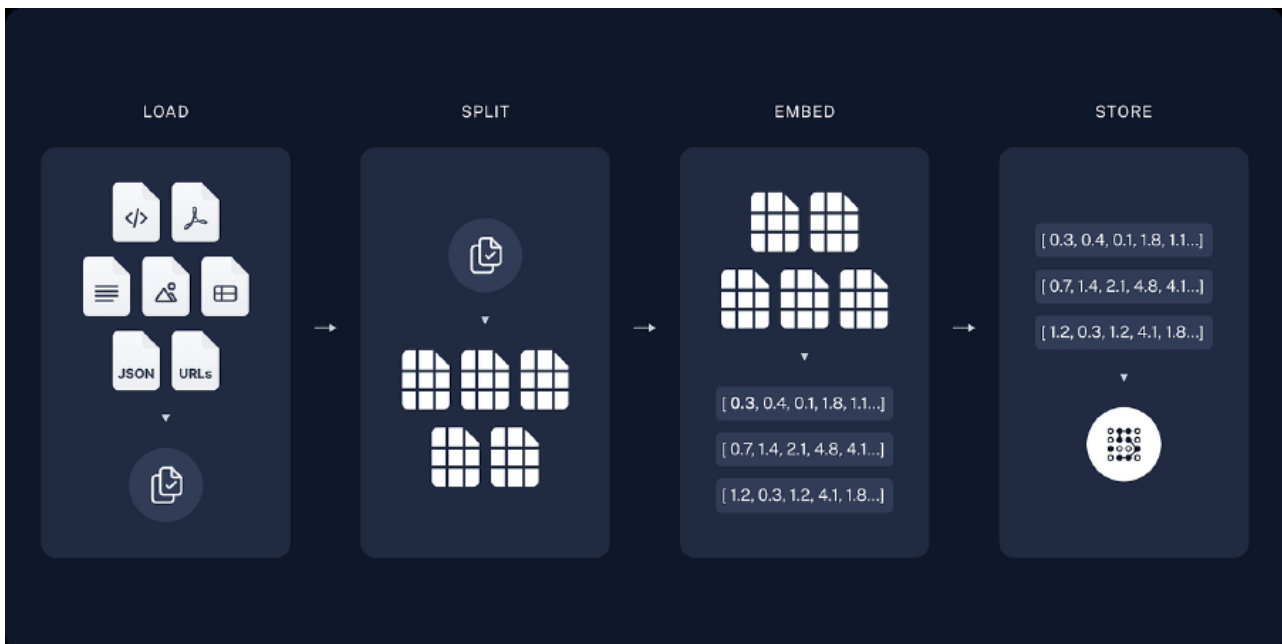
Share

## Vector Databases

Well, you may say we can store these in vector databases, thats right, but there are so many vector databases available in the market, which one would you go with ? Each one of them have their own pros and usecases. For a fact even if you pretend vector databases don't exist, a RAG system is still totally doable with just:

an array/tensor of embeddings, shape: (num\_chunks, embedding\_dim)

some metadata structure that tells you which row corresponds to which chunk of text.



No Vector DB Approach: - persistence to the disk, all embeddings must fit into memory and usually when lesser number of embeddings are involved

Vector DB Approach - db handles indexing, caching etc. - multiple services can read/write concurrently - millions of chunks and embedding vectors are involved.

The Vector DBs that are available in the market are: (chroma as well, missed to include that in the image)

# VECTOR DATABASES & LIBRARIES



## PINECONE

- Private, Cloud Based, Hugely Popular



## MILVUS

- An open-source similarity search engine for embedding vectors



## PGVECTOR

- Postgres Vector Search Extension, Useful if you already have data in Postgres



## WEAVIATE

- Open Source Vector Database written in Go



## FAISS

- A library for efficient similarity search and clustering of dense vectors



## QDRANT

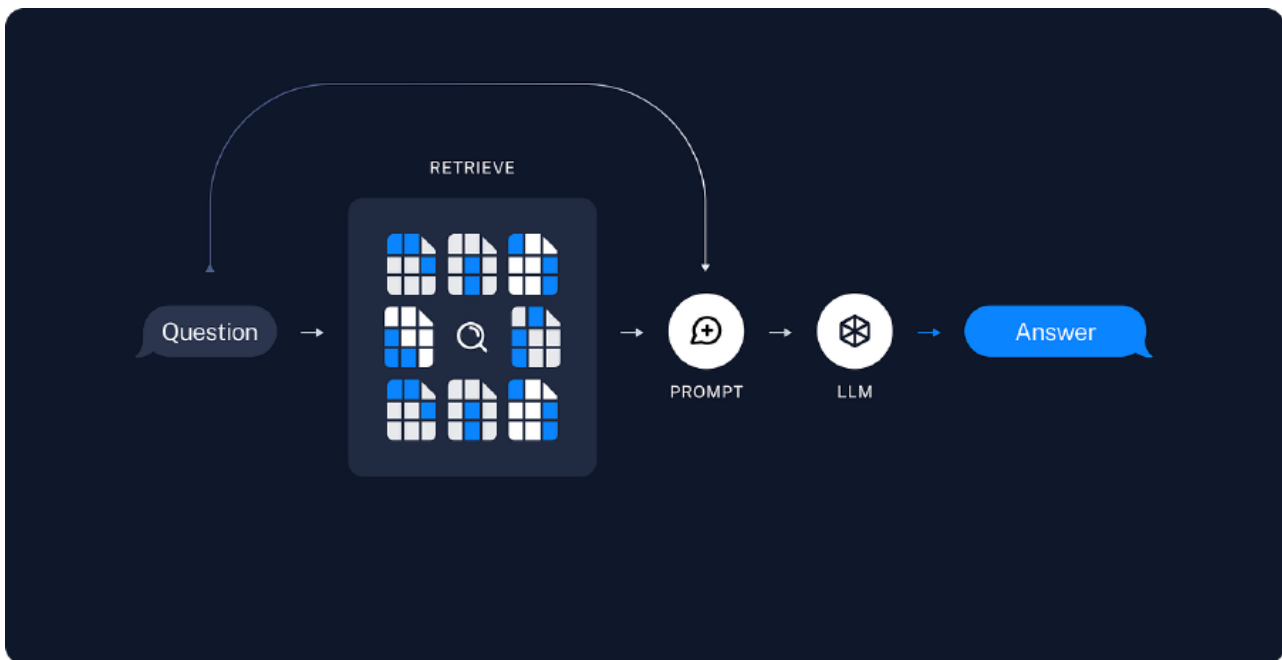
- Open Source Vector Search Engine written in Rust

Why do vector databases need indexing? Every time a user asks a question, we encode that query into an embedding vector on the fly. To answer the question, we need to find the most semantically similar vectors (chunks) from our stored corpus. If we naively compare the query embedding with every stored embedding one by one, it becomes very slow as the number of chunks grows.

Vector indexes solve this by organizing embeddings in a special data structure (HNSW graphs, IVFFlat, etc.) so that we can jump directly to the most promising neighbors instead of scanning everything. That's what makes similarity search fast enough to be usable in production RAG systems. I can talk about different indexing techniques in a different article, the better the indexing strategy, the better and faster the retrieval would be. By default, all of these databases come up with these indexing strategies where we can choose.

For clarity, the query embeddings are not stored anywhere; they are just calculated on the fly, just the document chunks' embeddings are stored in the Vector DB. The same model must be used for both the document and query embedding to ensure uniformity between the two.

## Retrieval



Now in the retrieval step :

The user sends a query.

You embed the query into a vector.

The system compares this query embedding with all the document chunks embeddings (via a vector DB indexing).

It retrieves the top-k(should be mentioned in our code) chunks whose embeddings are most similar to the query, using metrics like:

cosine similarity

Euclidean distance

(or other distance metrics supported by the vector store)

These retrieved chunks are considered to be the most relevant context for the user's question and are then passed into the Generation step, which we'll cover in Part 3, along with how to evaluate whether this whole RAG setup is actually doing a good job. I will also try to talk about the different indexing strategies used in these vector databases. Hope you liked this article. If you did, please do shower some love, and do subscribe to this weekly newsletter. Have a nice day! See you in the next one! ?

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

# Everything you need to know about production ready RAG systems : Part 3

2025-11-28 · Deep dive into the Generation phase in RAG, and widely used RAG evaluation strategies

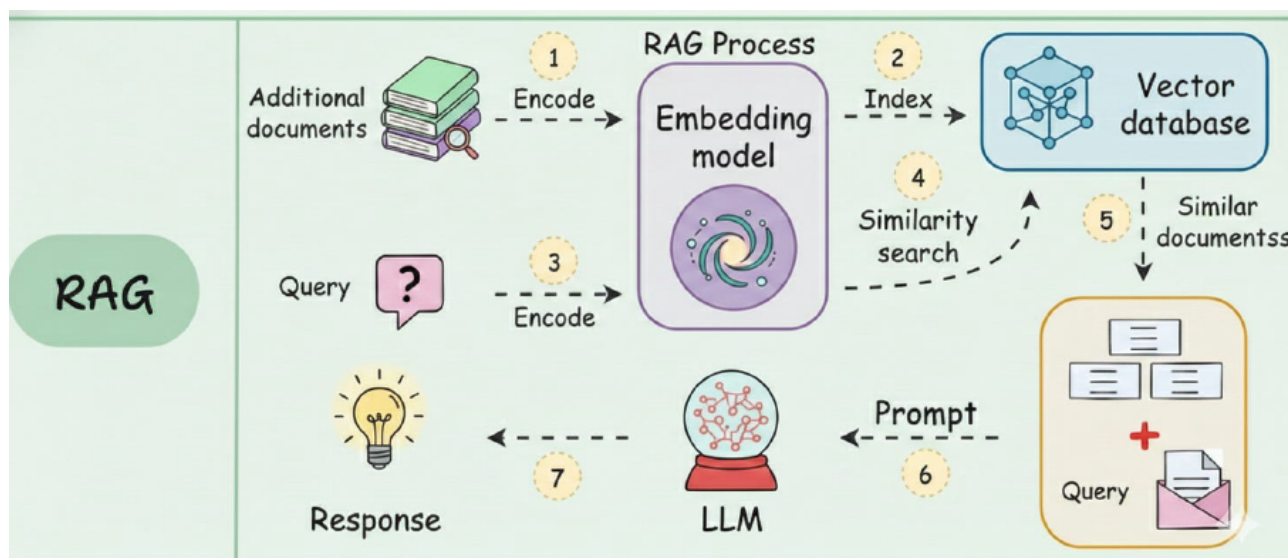
Source: <https://pradyumnachippigiri.substack.com/p/everything-you-need-to-know-about-9be>

In Part 1 and Part 2 we saw the Retrieval and Augmentation phase of RAG. Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/rag-systems-part3>

In this part, we will talk about the Generation part of RAG, along with ways to evaluate a RAG setup.

## Generation



By this point in the pipeline, we've already turned everything into vectors: we have an embedding for the user's query and embeddings for all our document chunks. Using similarity search, we've fetched the nearest neighbor document chunks, the pieces of text that are most likely to be relevant to the question. These retrieved chunks become the context window for the next stage: Generation.

In this Generation phase, the retrieved text chunks, along with the initial user query, are fed into a language model with a prompt. The algorithm will use this information to generate a coherent response to the user's questions through a chat interface. This integration allows the LLM to generate responses that are more accurate and contextually relevant, as it can draw from the most current and authoritative information available. This process ensures that the LLM has access to both the context provided by the user's query and the additional, relevant data fetched during the retrieval phase.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

## Choosing the Model: Local vs API

Here, as an engineer, you have to make a decision: what type of LLM are you going to use? There are two core questions you should ask yourself:

Do you want to run it locally?

If yes, how much compute and memory can you realistically dedicate to it?

If you want the best possible performance with minimal infra hassle, you'll likely start with a hosted API such as GPT or Claude. The trade-offs:

Pros: strong model quality, easy to integrate, no infra to maintain.

Cons: You're sending data to a third-party API (the data is leaving your system), and you pay per token.

If you want to run models locally (or in your own VPC), the second question becomes more important:

What hardware do you have (CPU/GPU, RAM)?

What latency and throughput do you need?

What's your budget for infra?

Your answer to these will determine whether you use a small/medium open-weight model locally, or a large hosted model via API.

## Why Prompting Matters in RAG

Writing effective prompts becomes a very important aspect in RAG as well because without effective prompting, even the most relevant retrieved information might not be used correctly by the LLM.

The key reasons prompting matters in RAG are:

**Avoiding Hallucinations:** This is perhaps the most critical role. A well-crafted prompt explicitly instructs the LLM to only use the information provided in the context, preventing it from fabricating details or drawing upon its general knowledge when the context is insufficient.

**Generating Custom Outputs:** Prompting allows you to define the desired behavior, personality, tone, and style of the LLM's response. Want a friendly, concise answer or a detailed, formal one? Prompting makes it possible.

**Setting Up Constraints and Rules:** You can use prompts to enforce specific rules, such as prohibiting the mention of certain topics or requiring a particular format (like bullet points or JSON)

Almost every generation prompt in a production RAG system is some combination of these 4 layers:

Role / System message -> "Who are you?"

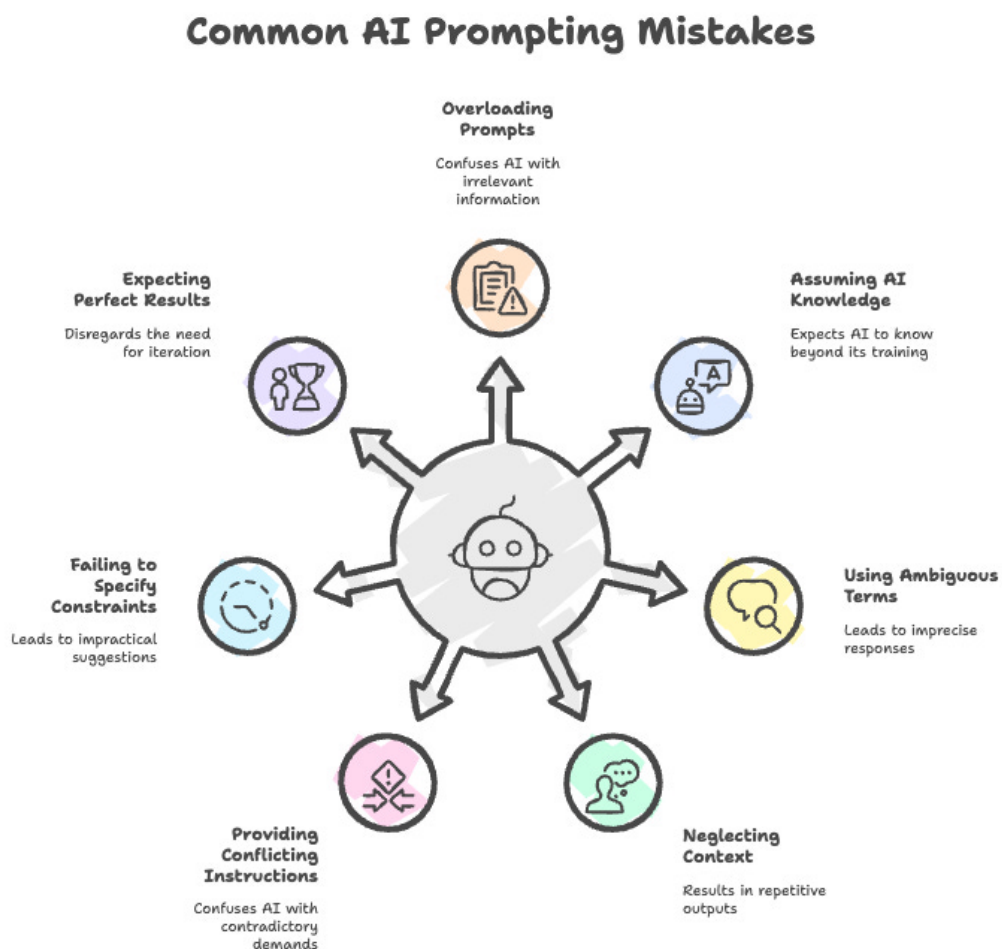
This is your guardrail layer. It's your best defense against hallucinations.

Task instructions -> "What should you do right now?"

Context -> retrieved chunks / "Here is what you're allowed to see" Helps in labelling the chunk for citing it later on if needed.

User query -> the actual question or request

It would be even better if you provide with 1/2 examples so that the LLM can understand better. Once a good prompt is prompted, the LLM outputs the required answers, and our RAG is now fully working to our documents and usecase. Some of the common mistakes in prompting you can see from the image below:



Made with Napkin

But we also need a way to test out this RAG setup of ours,

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

## **RAG Evaluation pipeline**

Evaluating a RAG pipeline helps in evaluating and understanding how our RAG is performing, it involves multiple steps which industries currently follow to evaluate their enterprise RAG.

Step 1: Build a small, high-quality test dataset

Before metrics, you need data:

Collect a set of questions your system should handle

For each question, store:

Ground-truth answer

Relevant source passages/document IDs

Even 30–100 good examples are enough to start. This becomes your RAG test set.

Step 2: Evaluate Retrieval – “Did we fetch the right context?”

Before you judge the final answer, you want to know: “Is my retriever even giving the model the right stuff to work with?”

So, for each question in the test set:

Run it through the retriever (but don’t call the LLM yet).

Collect the top-k chunks that come back.

Look at those chunks and ask:

If I were the model, could I answer this question from these chunks?

There are two ways to do this:

Manual check (great in the early stages): Skim a subset of questions and their retrieved chunks and judge them yourself:

“Yes, this chunk clearly answers the question.”

“No, this is just boilerplate/irrelevant.”

Automatic check (LLM-as-judge): Use another model to score relevance. For each (question, chunk) pair, you can ask: “Given this question and this chunk, how relevant is this chunk to answering the question? Score from 1 (not relevant) to 5 (highly relevant).”

This step isolates retrieval from generation:

If retrieval is weak, the LLM is basically guessing, no matter how good your prompt or model is.

If retrieval looks good but the final answers are still bad, that’s a strong signal that the problem is in the Generation step (prompt design, model choice, or output constraints), not in the retriever.

Step 3: Evaluate Generation – “Is the answer grounded & correct?”

Now we plug in the LLM and evaluate the answers themselves.

For each question:

Run full RAG: query -> retrieval -> generation.

Get the final answer.

Compare answer against:

The context (for groundedness).

The expected answer (if you have one).

Key dimensions:

Faithfulness / Groundedness - Does the answer stick to the provided context, or is it making stuff up?

Relevance - Does it actually answer the question?

Correctness - If you have a expected answers, how close is it?

Completeness - Does it cover all important parts of the query (not just one slice)?

Again, you can:

Do this manually at first,

Then automate with an evaluator model ("LLM-as-judge"), which you prompt like: "Given the question, the context, and the answer, rate:

Faithfulness: 1–5

Relevance: 1–5

Completeness: 1–5 And explain briefly why."

You can store these scores and take averages across the datasets or there are frameworks just for evaluation built for example like RAGAS.

Step 4: Measure Formatting & Contract Compliance

For production apps, it's not enough that the text is correct; it must also:

Return valid JSON when you ask for JSON.

Respect length constraints ("< 200 words").

Use the right tone or structure (e.g., always bullet points).

So you add checks like:

"Is the output valid JSON?" -> yes/no

"Does it contain all required fields?" -> yes/no

"Does it exceed X tokens?" -> yes/no

These can be fully automated with simple scripts.

Step 4: Close the loop and compare different configurations

Now the fun part: you can experiment and compare setups:

Embedding models: model A vs model B

Chunk sizes: 500 tokens vs 1000 tokens

Top-k: k=3 vs k=10

LLMs: GPT-4.x vs open-source model

Prompts: strict vs more relaxed

For each configuration, run:

Retrieval eval -> get relevance/recall metrics.

Generation eval -> get groundedness/correctness/completeness.

Formatting checks -> pass/fail.

In practice, you don't have to build all these evaluation metrics from scratch. There are dedicated frameworks such as RAGAS, TruLens, LangSmith, and Arize Phoenix that use LLM-as-judge-style evaluators to score both retrieval quality (context relevance/recall) and generation quality (faithfulness, answer relevance, hallucinations). RAGAS is one of the most popular RAG-specific libraries.

There are multiple evaluation frameworks out there along with numerous metrics you can read about them here - <https://www.pinecone.io/learn/series/vector-databases-in-production-for-busy-engineers/rag-evaluation/>.

This brings us to the end of the 3-part RAG series.

I hope you now have a clear picture of how RAG works in production systems: from ingestion and chunking, to retrieval and augmentation, to generation and evaluation.

I'd love to hear how you do it in your company, or if there's anything you think I missed.

Thank you so much for reading. Take care, you wonderful people and see you in the next one. ?

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

# A Supa Explanation of Supabase: The BaaS Everyone Is Talking About (And Why)

2025-11-12 · In this article we will understand what Supabase is, and why it is making so much positive noise in the market, and how it actually works behind the scenes.

Source: <https://pradyumnachippigiri.substack.com/p/a-supabase-explanation-of-supabase-the>

---

## What is Supabase?

There are many products in this space, if you've used Firebase or Appwrite, you already know the category: BaaS (Backend as a Service).

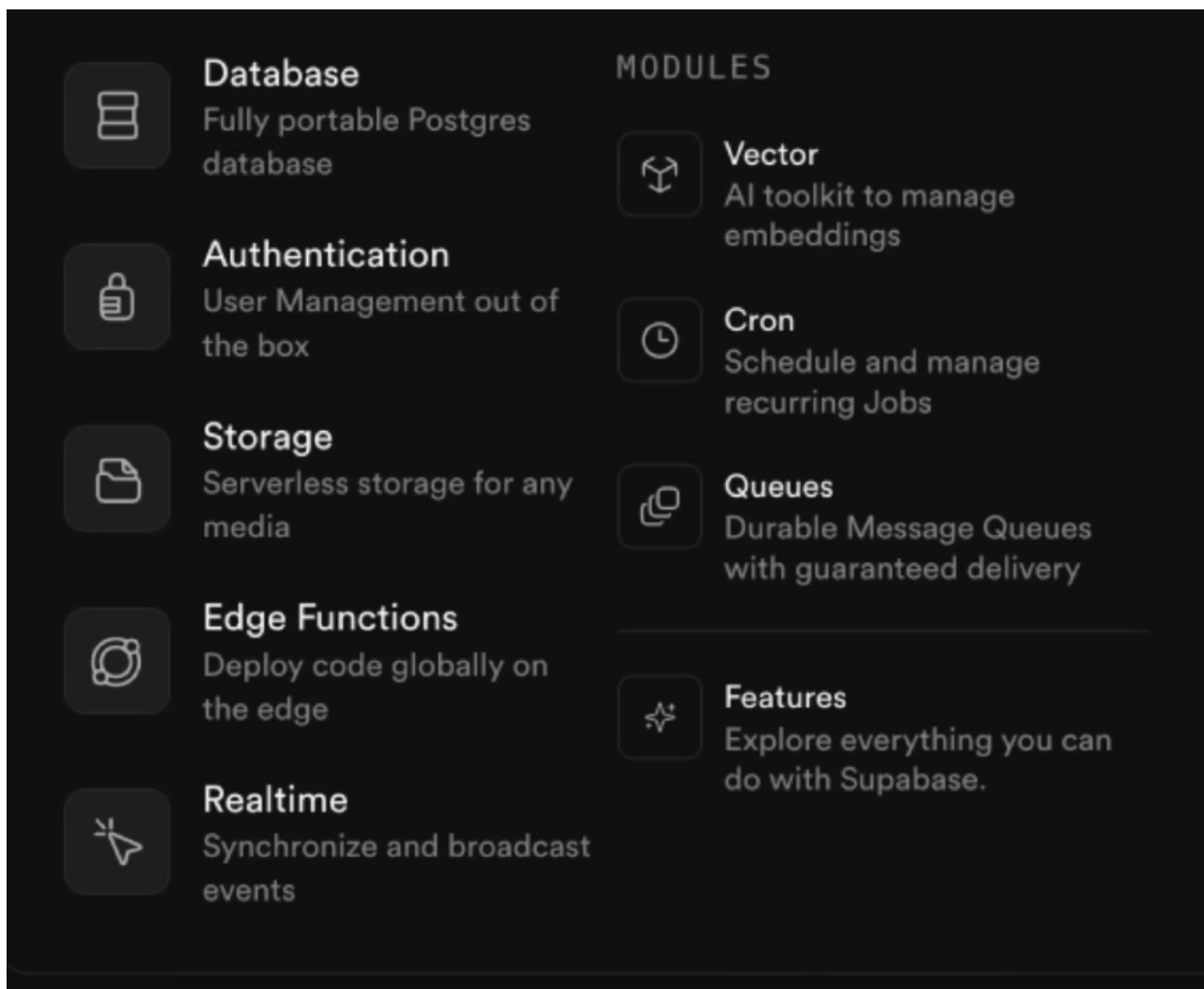
Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

When you build your first few projects, you eventually start seeing a pattern: the same backend pieces keep repeating like auth, database, storage, user management, file handling, etc. Every app needs them, and rebuilding those from scratch each time is exhausting.

That's why tools like Firebase, Appwrite, and Supabase exist. They package these common backend building blocks into ready-to-use APIs, so you can focus on your frontend instead of spinning up servers, databases, and auth systems manually.

This means you can build an entire application in React or Next.js and directly interact with the BaaS no custom backend server needed for most use-cases.

So Supabase is one of these BaaS platforms, but it stands out because it's built on top of PostgreSQL, is open-source, and gives you an SQL-first developer experience. And it handles scale enormously well.



In 2024, Supabase raised \$100M in Series E funding, reaching a valuation of around \$5B. That's massive. But what made this possible? Supabase embraced AI earlier than its competitors and positioned itself as the default backend for AI apps. With support for pgvector, embeddings, and real-time features on top of PostgreSQL, So supabase became extremely attractive for AI startups and developers building LLM-powered applications.

How Supabase works behind the scenes?

Let's say you're building a store app and want to query the name and price of products in the "electronics" category. Here's what your Supabase client code might look like:

```

import { createClient } from "@supabase/supabase-js";

const supabase = createClient(
  "https://[projectId].supabase.co",
  "public_key"
);

async function getProducts() {
  ...
  const { data, error } = await supabase
    .from("products")
    .select("name, price")
    .eq("category", "electronics");
  ...
}

getProducts();

```

This is the supabase query builder from the client-side library where we create the client by providing the project url and a secret key.

If we want to query data ("name" and "price") from the products table where products categorized as electronics.

At first glance, it feels like you're writing SQL right in your frontend but there's a lot happening under the hood.

Supabase converts this SQL like syntax of the client library code into a REST call with query parameters. Something like this :

This request first hits Supabase's API Gateway, which routes it to a service called PostgREST the magic layer that converts your REST query into real SQL for PostgreSQL to execute.

Here's what that REST call becomes internally:

On the backend, Supabase uses something called as the Prepared statements, which basically protects the SQL from SQL Injection meaning Supabase parameterizes it to prevent SQL injection attacks. Our constructed query is wrapped around a template like this :

Parameterized queries not only improve security but also allow for better caching and performance optimization.

### Handling Scale with Connection Pooling

If your app suddenly goes viral and thousands of users start querying at once, creating a new database connection for every request would slow things down fast.

That's where connection pooling comes in. Supabase uses pgBouncer, a lightweight PostgreSQL connection pooler, which keeps a pool of open connections ready to be reused. This ensures queries are handled efficiently no matter how much traffic spikes.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

### Enforcing Security with Row-Level Security (RLS)

Here's where things get interesting.

Now, before PostgREST sends the query to PostgreSQL for it to execute, there is one more important layer that comes in, and that is Row Level Security. Now what's this new thing that everyone's talking about?

The thing is, PostgREST first has to make sure that whoever is executing this query is actually allowed to read the data or not.

And as we discussed, BaaS does not have a backend as we tend to connect the frontend directly to Supabase. If you don't have any backend protection layer, what if data is not secure or gets leaked, especially when you try to talk to the DB directly from the frontend? That's where RLS comes in.

## What is Postgres Row Level Security?

Row level security (RLS for short) is a PostgreSQL security feature provided by the open source PostgreSQL database. It allows database administrators to define policies to control how specific rows of data display and operate for one or more user roles.

Row level security restricts database operations on a row level.

For example, the policy below states that only authenticated users can read the data from the table.

×

Create a new Row Level Security policy

Policy Name

Enable read access for all users

Table on clause

public.products

Policy Behavior as clause

Permissive

Policy Command for clause

☒ SELECT

☐ INSERT

☐ UPDATE

☐ DELETE

☐ ALL

Target Roles to clause

authenticated ×

🔒 USE OPTIONS ABOVE TO EDIT

```
1 create policy "Enable read access for all users"
2 on "public"."products"
3 as PERMISSIVE
4 for SELECT
5 to authenticated
6 using (
7   true
8 );
```

When an authenticated user calls `getProducts()`, their JWT token (from Supabase Auth or an external provider like Clerk) is sent along with the request. Supabase verifies that token before executing the query if the user isn't authorized, the query never runs.

Once the SQL query executes, PostgREST receives the result from Postgres, converts it into JSON, and returns it to your frontend. And on the frontend the data would be in the same structured format that you'd expect from any modern API.

| name           |  | price  |
|----------------|--|--------|
| -----+-----    |  |        |
| iPhone 13 Pro  |  | 845.99 |
| Sony XM4       |  | 199.99 |
| MacBook Air M1 |  | 899.99 |

```
[  
  { "name": "iPhone 15 Pro", "price": 845.99 },  
  { "name": "Sony XM4", "price": 199.99 },  
  { "name": "Macbook Air", "price": 899.99 }  
]
```

Supabase manages the complex backend layers for you from authentication and security to query translation and scaling while letting you stay close to SQL. No boilerplate, no infrastructure stress. You get all the power of PostgreSQL without managing servers, roles, or credentials manually.

No wonder it's rapidly becoming the go-to backend for SaaS and AI startups.

? [Explore the Supabase Documentation](#)

Thank you so much for taking the time to read this article. It takes effort to research, write, and break down complex systems like Supabase into something clear and practical, so if this helped you understand it even a little better, that means a lot.

If you liked it don't forget to share, like, and subscribe to my newsletter.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Stay curious, keep building, and I'll see you in the next article.

# Agent Memory Types

2026-01-18 · ai

Source: Engineering notes — pradyumnac26.github.io

The kind of data stored in an agent's memory typically falls into three buckets:

- Semantic memory (stable facts and preferences of the user)
- Episodic memory (storing specific past events/episode/decisions)
- Procedural memory (store reusable workflows/playbooks as to how agents should behave.)

| Memory Type             | Definition                                                    | Example                                                                 |
|-------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Semantic (What)</b>  | Stores stable facts and preferences that stay true over time. | <i>"User prefers Python over JavaScript."</i>                           |
| <b>Episodic (When)</b>  | Stores specific past events/episodes (what happened, worked). | <i>"Last time, we fixed the API timeout by increasing the DB pool."</i> |
| <b>Procedural (How)</b> | Stores rules/playbooks for how the agent should behave.       | <i>"When summarizing, keep it to 5 bullets max."</i>                    |

# How Agents write to external long term memory

2026-01-18 · ai

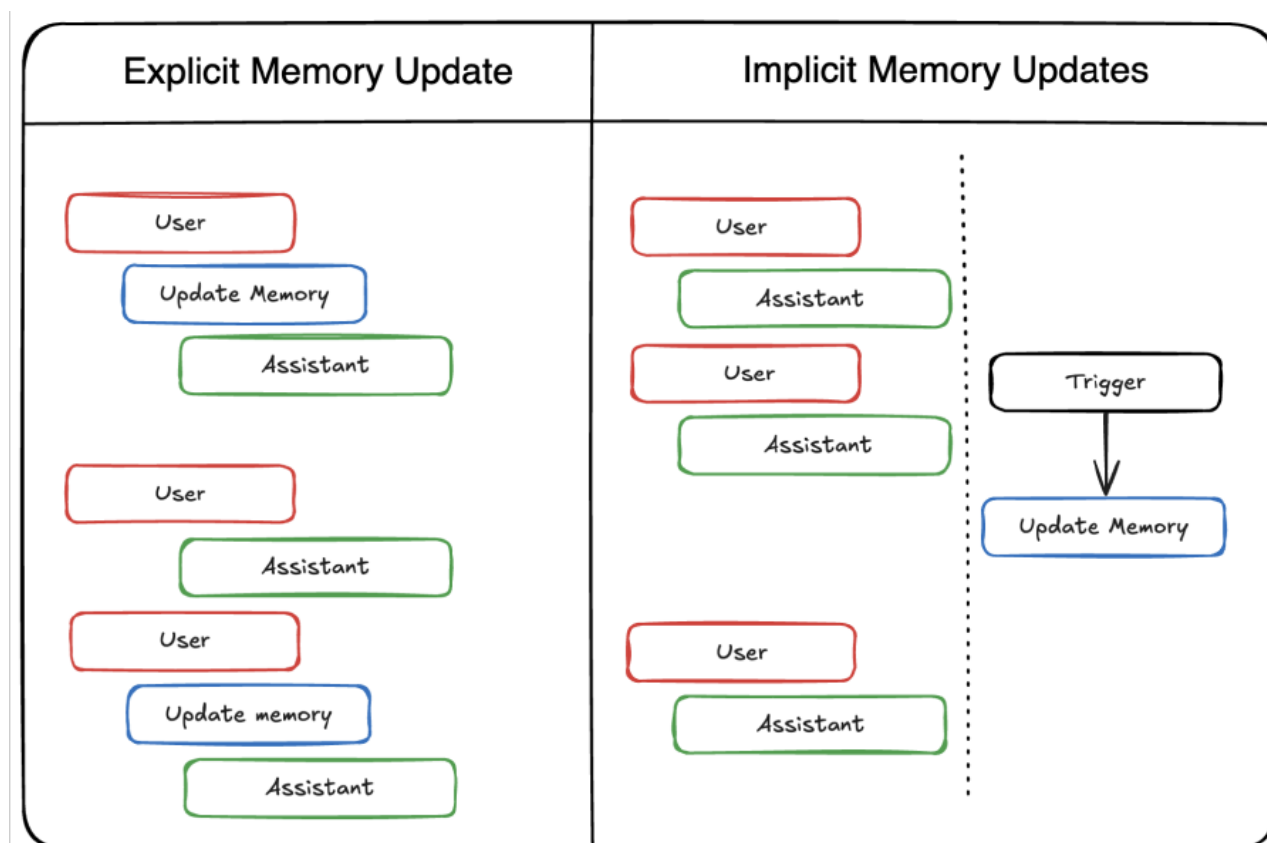
Source: Engineering notes — pradyumnac26.github.io

The agent system exposes a memory tool , and the model triggers it based on a memory policy.

When triggered, the system writes to an external store (vector DB) and later retrieves relevant bits back into the context window.

The 2 common write paths it can follow :

- Explicit write (hot path): The agent identifies that something is important during the conversation and saves it immediately using tool calling. This updates long-term memory right away, so it can be used in the next turn.
- Implicit write (background): The agent responds first, and a background process later summarizes/extracts useful facts and writes them to long-term memory either after a session or during periodic intervals, This avoids latency, but the memory may not be available instantly for the next message.



But what kind of data goes to long-term memory ?

It typically falls under these 3 buckets:

- Agent Memory Types (semantic, episodic, and procedural memory)

This kind of context memory management, by offloading context to long-term memory, helps prevent memory bloat and keeps the context window clean with relevant and sufficient data

# Prompt Injection

2026-09-18 · ai

Source: Engineering notes — pradyumnac26.github.io

---

Prompt injection isn't a prompt problem, it is an architectural problem.

Fixing prompts and making it better won't solve it because LLMs can't distinguish between instructions and data.

The only way we can reduce the blast radius is by :

- building the architecture assuming the LLM will leak, applying the principle of least privilege principle.
- Isolating and limiting what the LLM can access, so that if it leaks, nothing critical is exposed.
- the LLM should not be in charge of access control, let it request, let the backend decide based on user privileges
- Log everything in order to trace back in case of an incident.
- validate outputs using strict schema before using them.
- conduct attack simulations prior, so that we catch hold of these leaks prior to a real prod leak.

Best practice : Design assuming the model is compromised.

# Knowledge Distillation

2026-09-27 · ai

Source: Engineering notes — pradyumnac26.github.io

---

knowledge distillation or model distillation is the process of transferring knowledge from a large model to a smaller one

We already have a huge, smart model that's fully trained. But it's expensive at scale (costs a lot in compute and energy), needs heavy hardware, and can't fit on small devices, so for everyday use, we actually want a small, fast model: one that uses less compute and memory, responds quickly, and can run on smaller devices. Instead of training that small model from scratch on raw data, we have it copy the big model's predictions. This is knowledge distillation.

Google's Gemma models are distilled from Gemini. Meta's Llama 4 Scout and Maverick are distilled from the 2-trillion-parameter Behemoth model.

In 2006 published paper Model Compression, At the time, models were tiny by today's standards (maybe 500MB to 1GB), but the problem then was that the best performing models weren't single models at all. They were ensembles : collections of hundreds or thousands of independently trained classifiers, whose predictions were averaged together for a final answer.

Ensembles worked well because training back then was noisy and sensitive to random initialization. Train a thousand slightly different models on the same spam-detection task, and some come out great, some come out mediocre. Averaging all of them would give better results.

The problem back then was that a thousand-model ensemble cant fit into a device with limited storage. So the Cornell paper proposed something clever: instead of shipping the whole ensemble, train one small model to mimic the ensemble's averaged output

The small model was not trained only on the original hard labels from the dataset, such as "spam" or "not spam." Instead, it was trained to match the ensemble's averaged probability output for example,  $P(\text{spam}) = 0.73$  rather than simply  $\text{spam} = 1$  These soft labels contain more information about the ensemble's behavior, including how confident it is in each prediction. By learning these probability outputs, the small model is learning the behavior. This soft, probability-based target is what makes the whole idea work, and it's the seed of everything that came after

The 2006 idea sat mostly unused until 2015, Distilling the Knowledge in a Neural Network at Google. This paper is where the field got its name and its "teacher/student" terminology. It also made a discovery that goes beyond the 2006 work: you don't need an ensemble at all. A single well-trained teacher model is enough. If you already have one big classifier and want to compress it, you're still better off training the small student on the teacher's soft probability output than on the dataset's hard labels.

They tested this on MNIST, the standard handwritten-digit dataset (0–9).

Take a messy handwritten "5." A hard label says "5 = 100%, everything else = 0%," discarding any resemblance to other digits. The teacher's actual output before rounding to a final answer might look like: 5 -> 60%, 3 -> 30%, 8 -> 10%.

That 30% on "3" is information: it tells the student that this input resembles a 3 more than it resembles any other non-5 digit. So this paper called this dark knowledge: information about what an input is not, contained in the small soft non-zero probabilities that a hard label erases. The teacher has already worked out these resemblances; the student only has to copy them, which is why it can use far fewer parameters than the teacher and still perform well.

This distillation process can happen pre-training, or post-training or both. Llama does it during pre-training.

Google and Meta does Proper distillation - type of knowledge distillation

2 reasons why proper distillation is not used everywhere is that :

- it requires access to teacher model's actual raw Logits
- It is expensive : because u should build and maintain ur own teacher model first.

# Proper distillation - type of knowledge distillation

2026-09-27 · ai

Source: Engineering notes — pradyumnac26.github.io

---

Proper Distillation also called as the Standard Distillation is a type of Knowledge Distillation process.

Here, the student model starts as a completely untrained, randomly initialized network, same as any model before training begins.

Here's the sequence at step 1:

1. Student has random weights. Feed it a piece of text.
2. Its prediction is garbage, essentially random probabilities over the vocabulary, since it hasn't learned anything yet.
3. The teacher (already fully trained, frozen) is fed the same text and produces its own soft distribution, this one is meaningful, not random.
4. You compare the student's garbage output to the teacher's meaningful output, compute the loss, and backpropagate it through the student's neural network. The teacher's weights don't change; it's just being used to generate a target.
5. The student's weights shift a tiny bit toward matching the teacher.

Repeat this billions of times across the training corpus. Each step nudges the student a little closer to producing distributions like the teacher's. By the end, the student despite having far fewer parameters has gradually learned to approximate the teacher's behavior.

# Model parallelism

2026-10-02 · ai

Source: Engineering notes — pradyumnac26.github.io

---

We saw the 2 problems in scaling neural networks in this post

Distributed Data Parallelism solves scaling when data is the bottleneck, but it assumes the whole model fits on one GPU.

But what if the model itself is too big to fit ?

That's where Model Parallelism comes in.

For scenarios where the model is too large to fit on a single GPU, we need model parallelism to distribute the parameters across multiple GPU devices.

There are 2 main ways the models can be distributed :

- we can either distribute the various layers / blocks across different GPUs (pipeline parallelism) eg : GPU-1 has layers 1–5, GPU-2 has layers 6-10, data flows sequentially through GPUs, one layer's output feeds the next GPU.
- split individual layers themselves across GPUs (tensor parallelism) meaning instead of dividing the neural network by layer (horizontally), you divide the tensors themselves (vertically). E.g., GPU-1 has half the weight matrix of `_one_` layer, GPU-2 has the other half of the same layer, both GPUs work on the same layer simultaneously, each computing a slice of the weight matrix.

Note Model Parallelism doesn't imply that the training happens in parallel!

# Distributed Data Parallelism

2026-10-02 · ai

Source: Engineering notes — pradyumnac26.github.io

Distributed Data Parallelism (DDP) is one of the strategy to Scale Neural Network Training Across Multiple GPUs.

We saw the 2 problems in scaling neural networks in this post

Distributed Data Parallelism solves scaling when data is the bottleneck,

The core idea is simple: we want to process larger batches of data by spreading the work across multiple GPUs.

In practice, replicate the entire neural network model on each GPU, split the data batch across GPUs, and synchronize gradients.

The steps go like this :

- Split the training dataset into batches.
  - Divide that batch data into equal shards (one per GPU). Each shard data is a subset of that particular batch data, together they cover the full batch.
  - Define one neural network (architecture + initial weights)
  - Copy that neural network to every GPU so each has the same weights at the start of training
  - Place each data shard on different GPUs (Batch 0 -> GPU 0, Batch 1 -> GPU 1, ...).
  - All GPUs run forward pass at the same time on their respective shard data. Each gets its own loss / activations.
  - All GPUs run backward at the same time. Each computes local gradients from its shard (different data -> usually different local grads).
  - Gradient synchronization happens between the GPUs to exchange and combine gradients across GPUs (sum, then often average). After this, every GPU holds the same full gradient for all weights.
  - Weight update happens in parallel -> Every GPU applies the same optimizer step with those same gradients. All model copies stay identical.
- To keep the gradients synchronized in data parallel training, all-reduce communication primitive is used.

Core Idea: The batch splits across GPUs and repeat from step 5

*distributed data parallel mov (video — see the online note)*

- DistributedSampler (from torch.utils.data.distributed) is used to split the dataset so each GPU process gets its own unique set of rows, with no overlap between GPUs.
- DistributedDataParallel (from torch.nn.parallel, imported as DDP) DDP wraps your model so that gradients computed on each GPU get automatically synced and averaged across all GPUs during training.
- init\_process\_group and destroy\_process\_group (from torch.distributed) init\_process\_group sets up the communication link between all GPU processes at the start of training, so they can talk to each other. destroy\_process\_group closes that link cleanly at the end.

# Strategies for scaling neural networks across multiple GPUs

2026-10-01 · ai

Source: Engineering notes — pradyumnac26.github.io

Training a neural network means repeatedly computing gradients and updating weights, batch by batch, until the loss converges.

But there are some problems :

- Training takes too long (too much `_data_` to process on one GPU in reasonable time)
- The model itself doesn't fit in one GPU's memory at all

## Why does memory run out in the first place

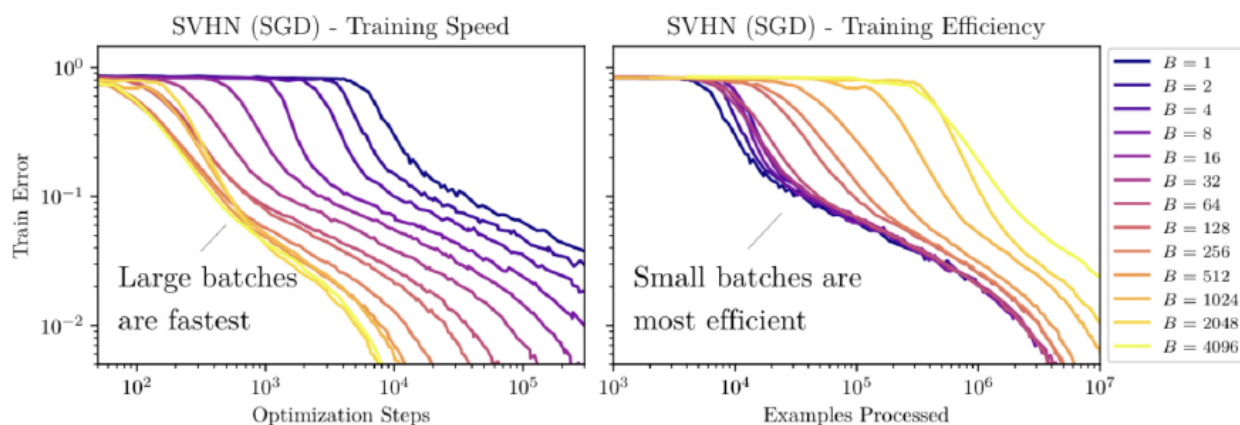
While training, a GPU has to simultaneously hold several things in memory, all at once:

- Model parameters : the learnable weights of the model
- Optimizer states : Optimizer states are extra values an optimizer keeps around to do its job, and exactly what gets stored depends on which optimizer is used. AdamW, for example, tracks first and second momentum estimates per parameter, on top of the parameters themselves.
- Model activations : intermediate values from the forward pass, kept around because backpropagation needs them to compute gradients.
- Gradients : stored for each parameter of the model, same memory footprint as the model parameters.
- Input data : the current batch being processed, the memory footprint depends on the size and type of data being modeled.

So if the total memory of all these things add up to be more than the GPU memory then thats when we see the "CUDA out of memory" exception.

Now the question becomes what if a smaller batch size can be used to reduce memory. But it comes with a trade off. Batch size affects 2 different things. It is talked about in this paper [An Empirical Model of Large-Batch Training](#)

- Smaller Batches -> Cheaper to compute and is fast, but the gradient estimate becomes noisier (not reliable), and it would need more steps to reach a good result.
- Larger Batches -> gives a smoother, more accurate gradient estimate, but costs more memory and compute per step.



There are some tricks that can reduce how much memory training uses on a single GPU. But these tricks have limits and only come up to a certain point, and they usually come with a catch: saving memory often means doing more computation instead.

So the only way to scale the neural network is by using multiple GPUs to our advantage.

There are three main ways to split up model training so it can run across multiple machines :

- Distributed Data Parallelism
- Model parallelism
- Fully Sharded Data Parallelism

# LLM Compute constraints

2026-09-18 · ai

Source: Engineering notes — pradyumnac26.github.io

Sources :

- <https://www.maartengrootendorst.com/blog/quantization/>

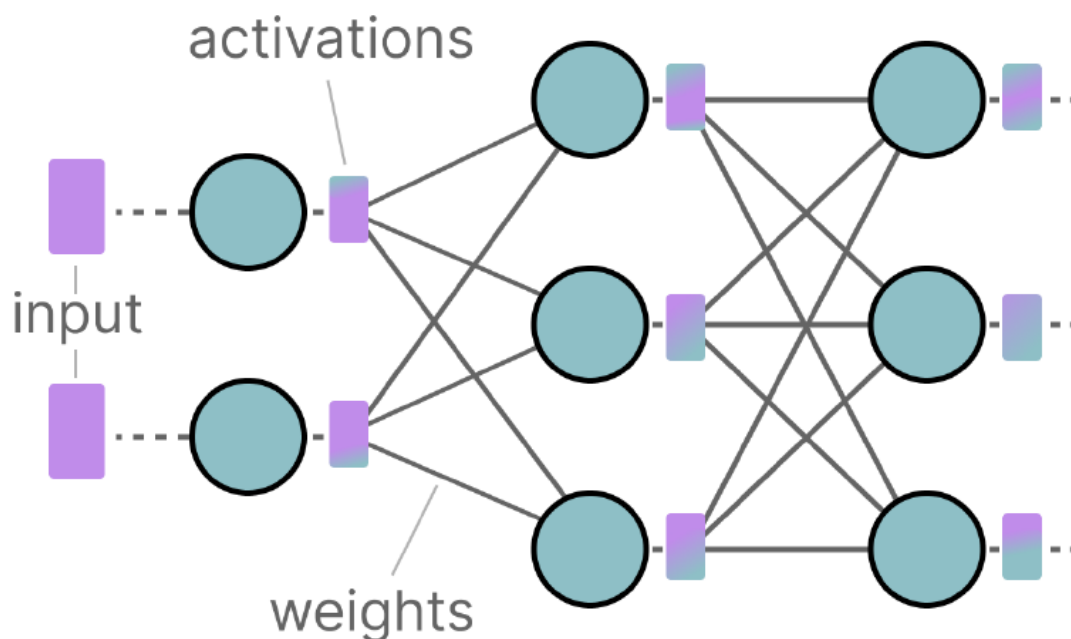
Large Language Models (LLMs) are often too large to run on consumer hardware. These models may exceed billions of parameters and generally need GPUs with large amounts of VRAM to speed up inference

As such, more and more research has been focused on making these models smaller through improved training, adapters, etc. One major technique in this field is called quantization

## The Problem with LLMs

LLMs get their name due to the number of parameters they contain. Nowadays, these models typically have billions of parameters (mostly weights ) which can be quite expensive to store.

During inference, activations are created as a product of the input and the weights, which similarly can be quite large.



As a result, we would like to represent billions of values as efficiently as possible, minimizing the amount of space we need to store a given value.

These values are represented by \*bits

A nifty feature of these bits is that we can calculate how much memory your device needs to store a given value. Since there are 8 bits in a byte of memory, we can create a basic formula for most forms of floating point representation.

$$\text{memory} = \frac{\text{nr\_bits}}{8} \times \text{nr\_params}$$

Now let's assume that we have a model with 70 billion parameters. Most models are natively represented with float 32-bit (often called full-precision ), which would require 280GB of memory just to load the model.

$$\mathbf{64\text{-bits}} = \frac{64}{8} \times 70\text{B} \approx \mathbf{560 \text{ GB}}$$

.....

$$\mathbf{32\text{-bits}} = \frac{32}{8} \times 70\text{B} \approx \mathbf{280 \text{ GB}}$$

.....

$$\mathbf{16\text{-bits}} = \frac{16}{8} \times 70\text{B} \approx \mathbf{140 \text{ GB}}$$

As such, it is very compelling to minimize the number of bits to represent the parameters of your model (as well as during training!). However, as the precision decreases the accuracy of the models generally does as well.

We want to reduce the number of bits representing values while maintaining accuracy... This is where quantization comes in!

# Logits

2026-09-26 · ai

Source: Engineering notes — pradyumnac26.github.io

Given a probability  $p$ , which ranges from 0 to 1, the corresponding odds can also be calculated as :

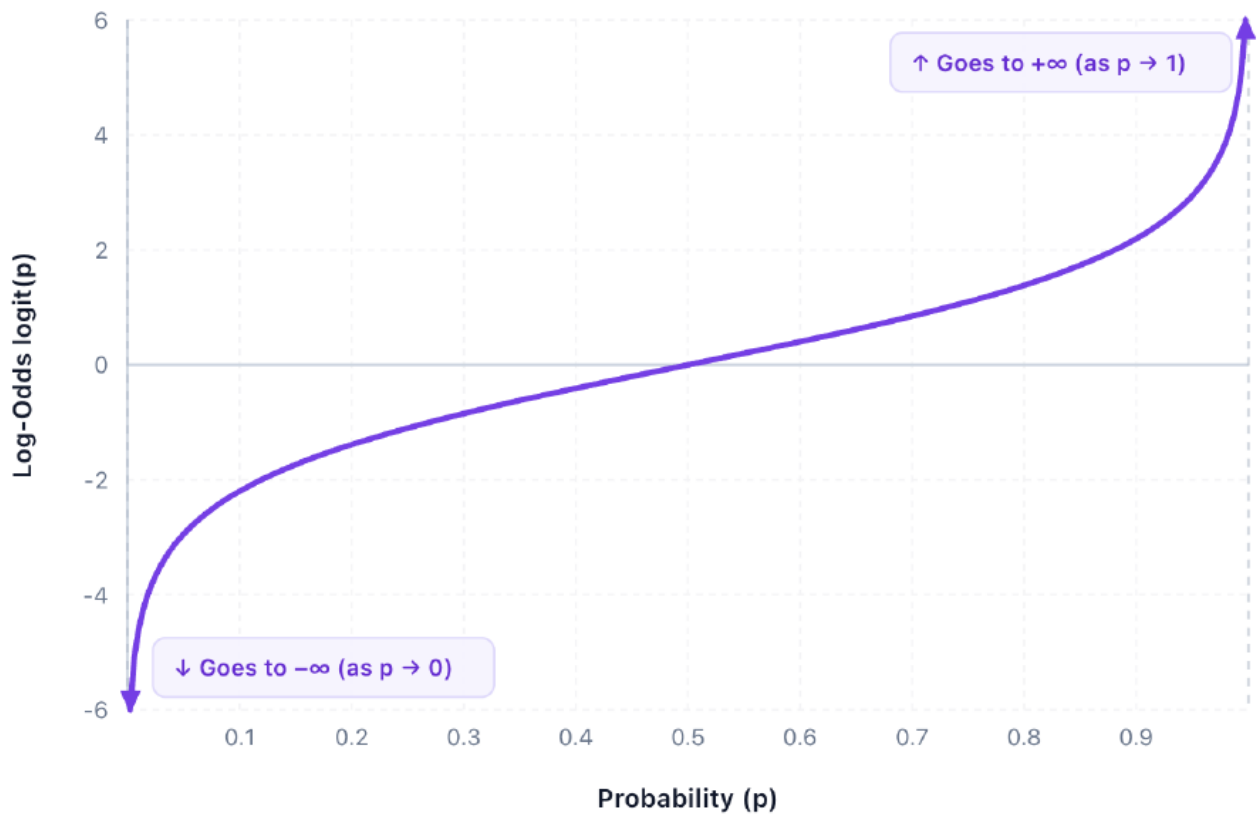
$$\text{odds} = \frac{p}{1-p}$$

meaning,

- if  $p > 0.5$ , then the odds are greater than 1, so success is more likely. for example consider  $p = 0.75$ , then the odds are 3 meaning you have a 3:1 favour so success is more likely.
- if  $p = 0.5$  then odds = 1, meaning there is a equal chance of success and failure.
- if  $p < 0.5$ , then odds are less than 1, so success is less likely

The logit function is simply the logarithm of the odds:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$



This transformation maps probabilities (0,1) to the entire real number range  $(-\infty, \infty)$ . The value of the logit function heads towards infinity as  $p$  approaches 1 and towards negative infinity as it approaches 0.

It is heavily used in Logistic regression models, where the raw outputs / predictions (which are in an unconstrained range) need to be converted into probabilities. So we consider the raw output of the logistic regression model as the logit.

So how do we convert this into a probability? That's where Sigmoid activation function comes in, it's the inverse of the logit function, and it maps that unconstrained logit back into the valid 0–1 probability range.

# Vanishing Gradient

2026-09-26 · ai

Source: Engineering notes — pradyumnac26.github.io

This is a very common problem when dealing with neural networks.

When dealing with Sigmoid activation function, one of the drawbacks is that it has vanishing gradient effect.

As we all know when a neural network learns, it works backwards (backward propagation) from output to input layer by layer. And during its backward propagation it learns the weights, parameters etc. based on the loss / gradient right.

Derivative of the Logits function:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = \log(p) - \log(1-p)$$

Differentiating term by term:

$$\frac{d}{dp} \text{logit}(p) = \frac{1}{p} + \frac{1}{1-p} = \frac{(1-p) + p}{p(1-p)} = \frac{1}{p(1-p)}$$

Derivative of the sigmoid function:

$p = \sigma(z) = \frac{1}{1+e^{-z}} \rightarrow \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$  Since  $\sigma(z) = p$ , we can write this more simply as:

$$\sigma'(z) = p(1-p)$$

As, we found:

$$\frac{d}{dp} \text{logit}(p) = \frac{1}{p(1-p)}$$

Compare that to what we just got:

$$\sigma'(z) = p(1-p)$$

These two are exact reciprocals of each other one is  $p(1-p)$ , the other is  $\frac{1}{p(1-p)}$ . This isn't a coincidence, and it isn't specific to logit and sigmoid, it's a general calculus rule that applies to any pair of inverse functions.

Now, when backprop works its way back to a weight  $w$ , it's not computing  $\frac{\partial L}{\partial w}$  directly in one shot it breaks it apart using the chain rule, since  $w$  affects the loss only through a chain of intermediate steps ( $w \rightarrow z \rightarrow p \rightarrow L$ ) because :

- $w$  changes  $\rightarrow$  this changes  $z$  (since  $z = wx+b$ )
- $z$  changes  $\rightarrow$  this changes  $p$  (since  $p = \sigma(z)$ )
- $p$  changes  $\rightarrow$  this changes  $L$  (since  $L$  is computed from  $p$ )

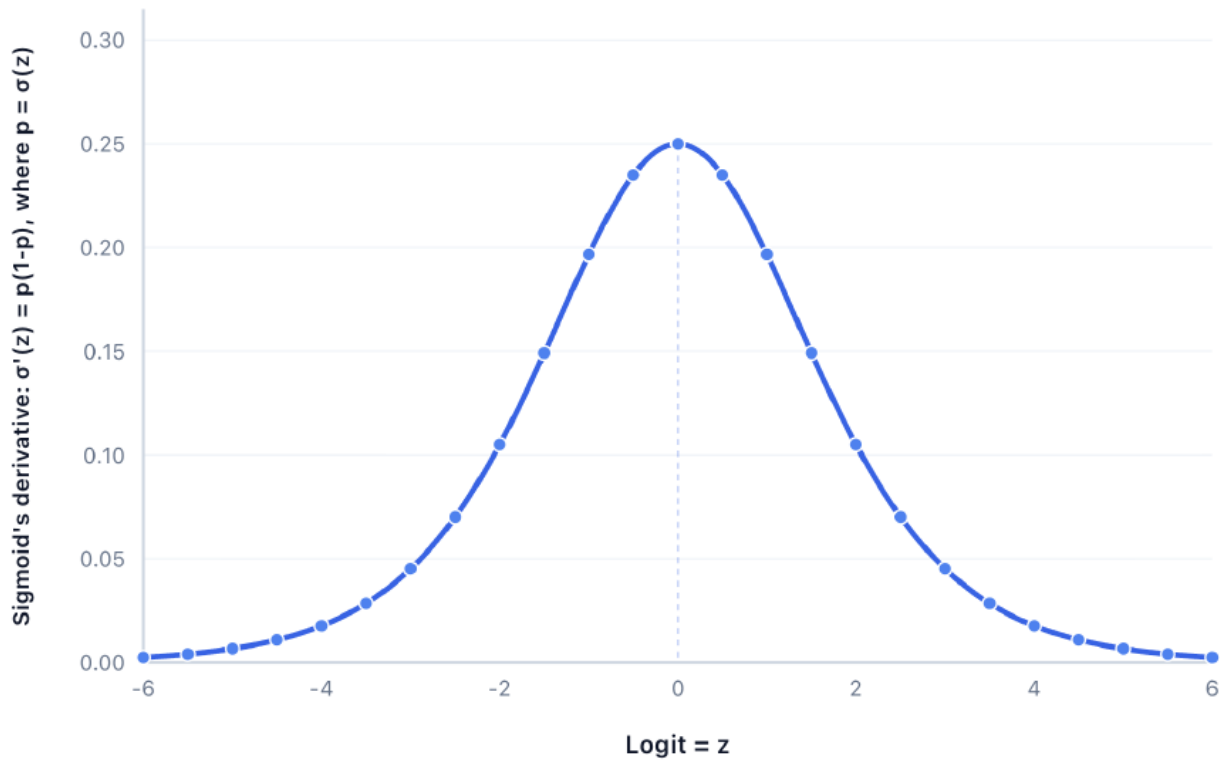
$\frac{\partial L}{\partial w} = \underbrace{\frac{\partial L}{\partial p}}_{\text{loss w.r.t. prediction}} \cdot \underbrace{\frac{\partial p}{\partial z}}_{\text{sigmoid's derivative}} \cdot \underbrace{\frac{\partial z}{\partial w}}_{=x}$  The middle term  $\frac{\partial p}{\partial z} = p(1-p)$ , the sigmoid's derivative is the one we need to look closely at, because it's the piece that misbehaves.

## Plotting it out

Let's compute  $p(1-p)$  at a few points along  $z$  (the logit) and see what happens:

| $z$ | $p = \sigma(z)$ | $\sigma'(z) = p(1-p)$ |
|-----|-----------------|-----------------------|
| 0   | 0.500           | 0.250                 |

|   |       |       |
|---|-------|-------|
| 1 | 0.731 | 0.197 |
| 2 | 0.881 | 0.105 |
| 3 | 0.953 | 0.045 |
| 4 | 0.982 | 0.018 |
| 6 | 0.998 | 0.002 |
|   |       |       |

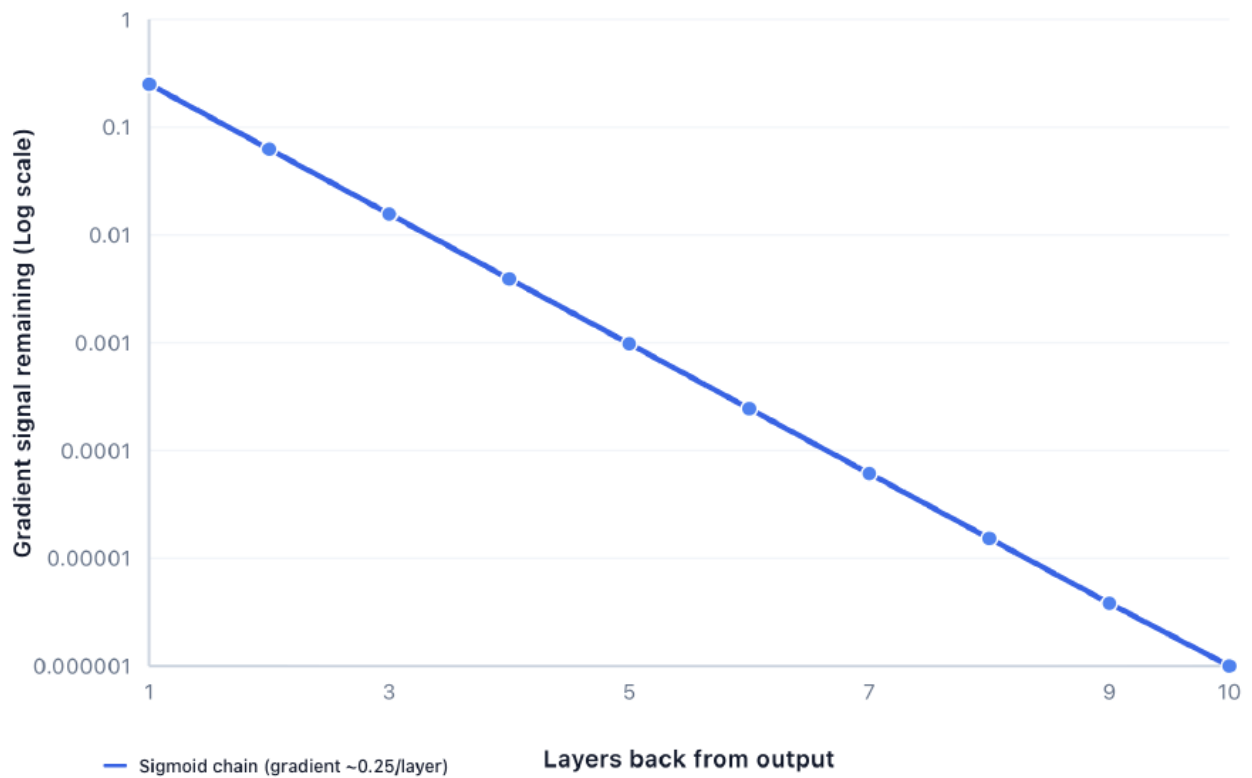


Look at the shape: it peaks at just 0.25 at  $z=0$ , and rapidly flattens toward 0 as  $z$  moves toward either  $-6$  or  $+6$ .

This curve above shows the behavior of a single neuron, at a single layer. It tells that if this neuron's logit is far from 0 (i.e., around -6, 6 or more), then its local gradient is tiny. hence this is called saturation.

As the number of layers increase, say if there are 10 layers then :

$0.25 \times 0.25 \times 0.25 \times 0.25 \times 0.25 \approx 0.00098$



Vanishing gradient is what happens when you chain many of these together across depth. Backprop multiplies the local gradients of every layer together as it works backward. So that network stops learning, or learns extremely slowly.

# ReLU activation function

2026-09-27 · ai

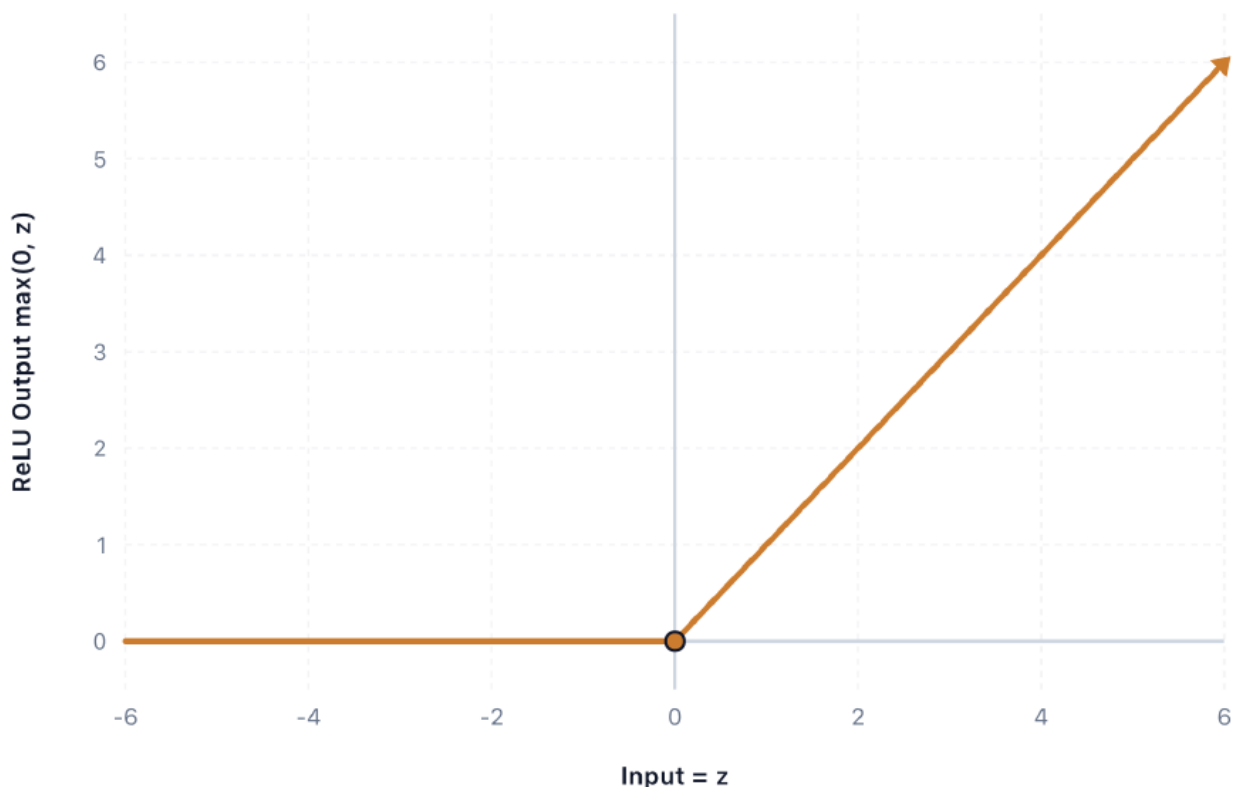
Source: Engineering notes — pradyumnac26.github.io

ReLU is called as the Rectified Linear Unit.

Just like Sigmoid activation function and Tanh activation function, ReLU takes the raw logit  $z = wx+b$  and transforms it into something else. But unlike those two, it doesn't squash anything into a bounded range.

$$\text{ReLU}(z) = \max(0, z)$$

- if  $z < 0$  : then output is 0
- elif  $z > 0$  : output is  $z$



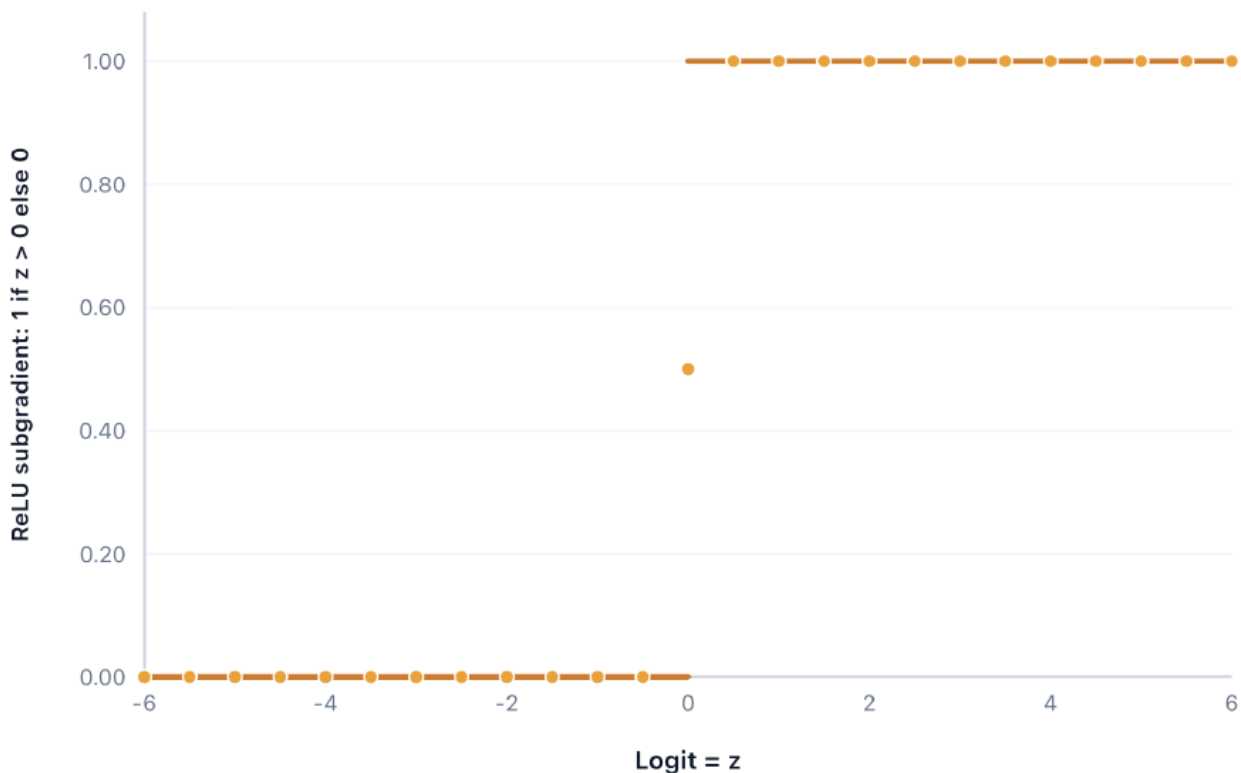
Since ReLU is defined piecewise, we differentiate each piece separately.

For  $z < 0$ : the function is just the constant  $0$ . The derivative of a constant is always  $0$ .

$$\text{ReLU}'(z) = 0 \quad \text{for } z < 0$$

For  $z > 0$ : the function is  $\text{ReLU}(z) = z$ . The derivative of  $z$  with respect to  $z$  is  $1$ .

$\text{ReLU}'(z) = 1 \quad \text{for } z > 0$  at  $z = 0$  ReLU is not defined, and so derivative also will be undefined. Hence there will be a sharp jump from 0 to 1 in the derivative graph.



So the derivative is either 0, or 1. The advantages of this is that :

- There wont be any Vanishing Gradient effect.
- So when backprop is happening we are multiplying repeatedly with 1s, so learning is faster as it descends quickly in the gradient descent process.
- It is cheap to compute as computers are super fast at handling 0s and 1s, instead of computing  $e^{-z}$  or  $e^z$  for every neuron like how Tanh and Sigmoid does.

One drawback of this activation function is that :

Once  $z < 0$  for everything,  $\text{ReLU}'(z) = 0$ . Now look at the chain again:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \underbrace{\frac{\partial a}{\partial z}}_{=0} \cdot \frac{\partial z}{\partial w} = 0$$

The gradient itself is 0. So the weights never get updated, irrespective of the learning rate  $\eta$ . So learning never happens

$$w_{\text{new}} = w_{\text{old}} - \eta \times 0 = w_{\text{old}}$$

## The clean summary of what you figured out

$$\eta \rightarrow \text{large -ve } w \rightarrow z < 0 \text{ for all inputs} \\ \rightarrow \frac{\partial a}{\partial z} = 0 \rightarrow \frac{\partial L}{\partial w} = 0$$

This problem is also called as the Dying Relu Problem. But with a proper setting of the learning rate this is less frequently an issue.

Thats why Leaky ReLu was introduced.

# Sigmoid activation function

2026-09-26 · ai

Source: Engineering notes — pradyumnac26.github.io

The sigmoid function is basically just the inverse of Logits. The sigmoid function maps arbitrary real values back to the range  $[0, 1]$ . The larger the value, the closer to 1 you'll get.

From the above, we can derive:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

$$e^{\text{logit}(p)} = \frac{p}{1-p}$$

$$e^{-\text{logit}(p)} = \frac{1-p}{p}$$

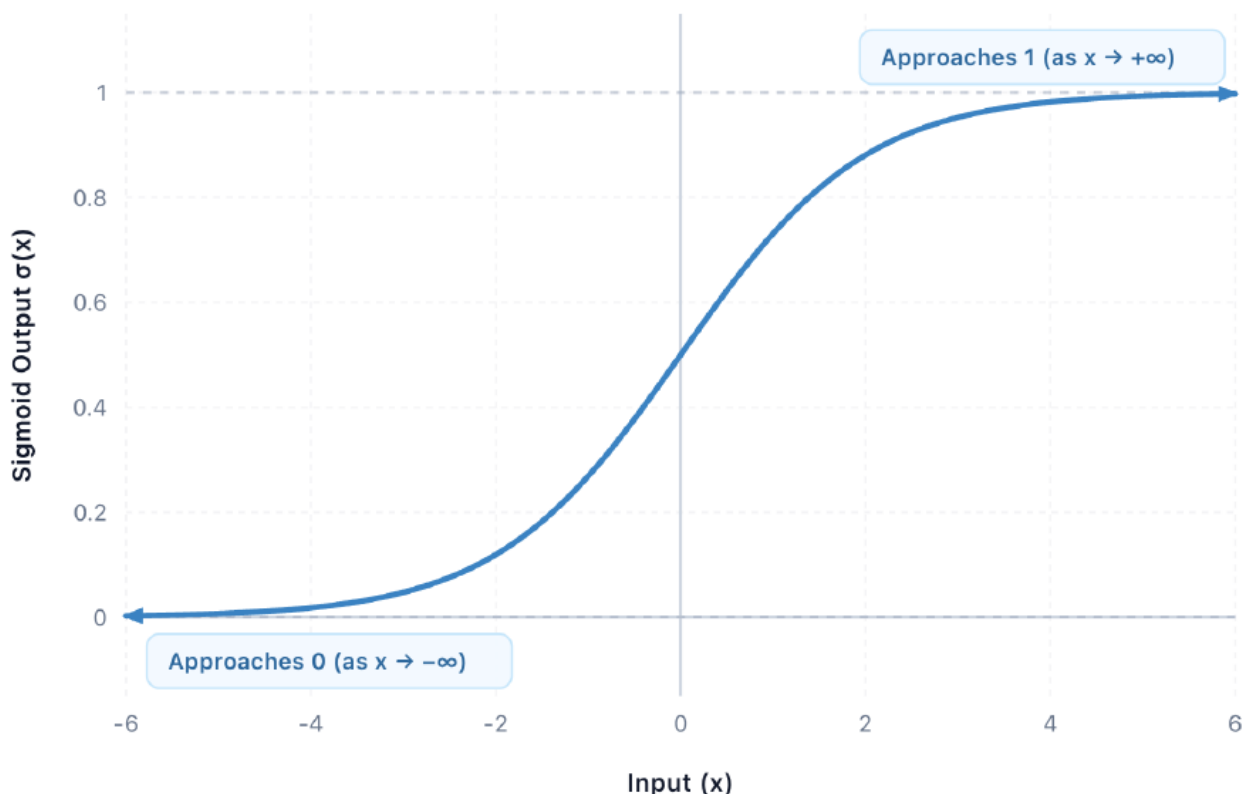
$$e^{-\text{logit}(p)} = \frac{1}{p} - \frac{p}{p} = \frac{1}{p} - 1$$

$$\frac{1}{p} = 1 + e^{-\text{logit}(p)}$$

$$p = \frac{1}{1 + e^{-\text{logit}(p)}}$$

$$p = \sigma(z) = \sigma(\text{logit}(p)) = \frac{1}{1 + e^{-\text{logit}(p)}} = \frac{1}{1 + e^{-z}}$$

where  $\sigma$  is the sigmoid function.



This is exactly what happens at the end of classification process like Logistic regression.

There are other functions that map probabilities to reals (and vice-versa), so what's so special about the logit and sigmoid? the gradients of the logit and sigmoid are simple to calculate. The reason why this is important is that many optimization and machine learning techniques make use of gradients, for example when estimating parameters for a neural network.

The biggest drawback of the sigmoid function for many analytics practitioners is the so-called vanishing problem.

this problem pertains not only to the sigmoid function, but any function that squeezes real values to the  $[0, 1]$  range. In neural networks, this Vanishing Gradient is often a problem. So you can find some alternatives.

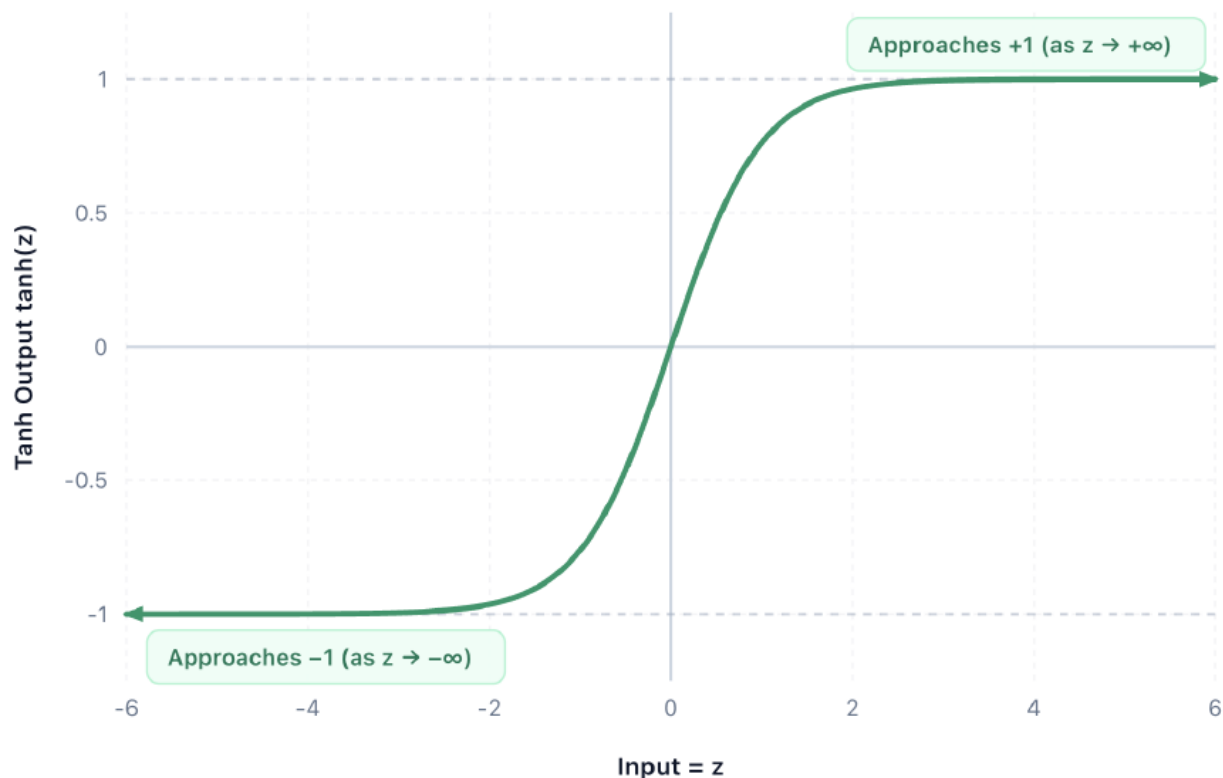
# Tanh activation function

2026-09-27 · ai

Source: Engineering notes — pradyumnac26.github.io

Tanh is hyperbolic tangent which is

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

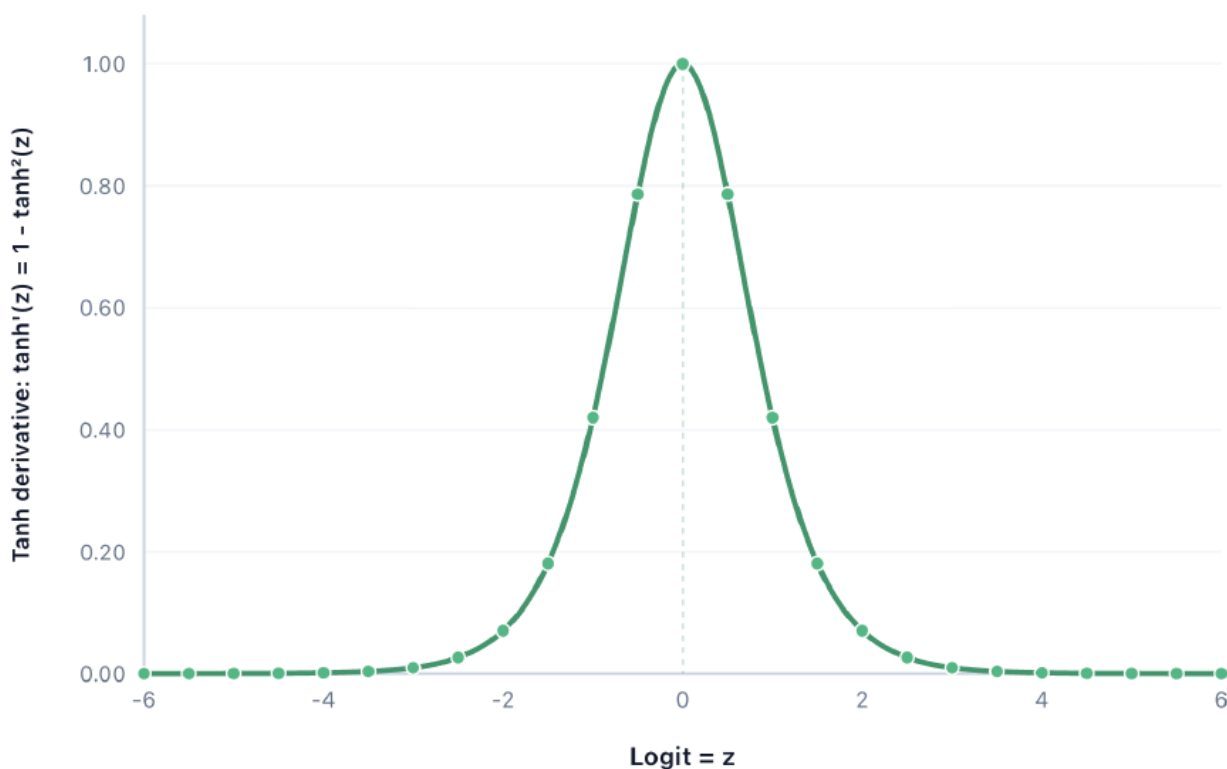


Same S-shape as sigmoid, but notice the range: -1 to 1, not 0 to 1.

So it squashes number in any real range  $(-\infty, \infty)$  to  $(-1, 1)$

Its derivative is :

$$\tanh'(z) = 1 - \tanh(z)^2$$



Like the sigmoid neuron, its activation also saturates hence causing Vanishing Gradient . but by  $z=4$ , the derivative has dropped to about 0.0013; by  $z=6$ , it's around 0.0000246 and then essentially zero, just like sigmoid.

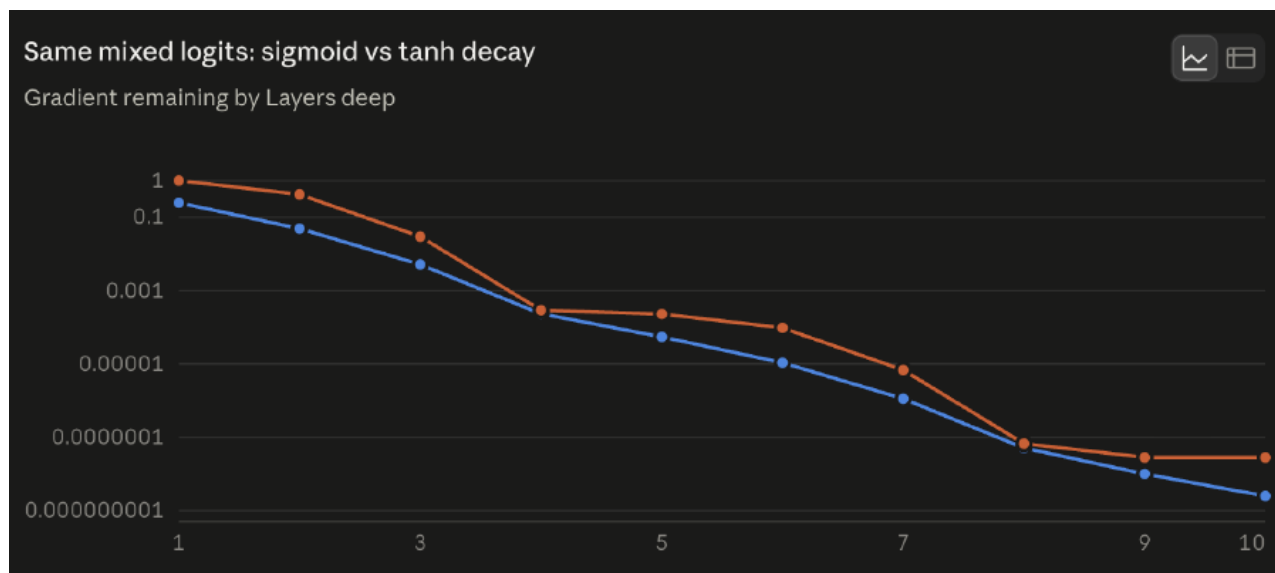
---

this peaks at 1.0 (at  $z=0$ ) instead of Sigmoid activation function 0.25. So this has approx 4x stronger gradient right at the center.

How is this centering around 0 useful ? weights tend to explore both positive and negative territory more naturally, rather than being biased in one direction from the start. As tanh is centered around 0.

How is the derivative being 1 at  $z = 0$  useful ? Because it means in the best case, a tanh layer doesn't shrink the gradient at all, and hence this decay happens slowly, means it learns faster.

As you can see from the graph below, it shows how sigmoid and tanh decays for the same  $z$ .



Also note that the tanh neuron is simply a rescaled and shifted sigmoid, in particular the following holds:  $\tanh(x) = 2\sigma(2x) - 1$